# CMR INSTITUTE OF TECHNOLOGY

**UGC AUTONOMOUS**

**Approved by AICTE, Permanent Affiliation to JNTUH & NBA Accredited**

**Kandlakoya (V), Medchal Dist-501 401**

**www.cmritonline.ac.in**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

## IOT AND CLOUD COMPUTING LAB MANUAL




**Prepared by**

**Mrs.G.Sangeetha**

**Asst. Professor, Dept. of CSE**

**Mrs.V.Sree Devi**

**Asst. Professor, Dept. of CSE**

**Mrs.G.Lavanya**

**Asst. Professor, Dept. of CSE**

**Mr.C..Kannan**

**Asst. Professor, Dept. of CSE**

**CMR INSTITUTE OF TECHNOLOGY**

Kandlakoya(V), Medchal Road, Hyderabad – 501 401

Ph. No. 08418-222042, 22106 Fax No. 08418-222106

**2025-26**

## IOT and Cloud Computing Lab

| | L | T | P | C |
|---|---|---|---|---|
| **III-B.Tech.-II-Sem.** | | | | |
| **Subject Code: 22CSPC64** | **0** | **0** | **2** | **1** |

**Course Outcomes:** Upon completion of the course, the students will be able to

1. Identify various IoT devices.
2. Use IoT Devices in various applications
3. Develop automation work-flow in IoT enabled cloud environment
4. Take part in practicing and monitoring remotely
5. Make use of various IoT Protocols in Cloud

**List of Experiments**

1. Perform necessary Software Installation for Arduino and Raspberry Pi.
2. Study of Arduino and Raspberry Pi boards and Install IDE for Arduino.
3. Write a program to transfer sensor data to smart phone using Bluetooth on Arduino.
4. Study and Implement RFID using Arduino.
5. Write a program to monitor temperature and humidity using Arduino and Raspberry Pi.
6. Write a program to monitor temperature and humidity using IoT Cloud application.
7. Upload temperature and humidity data to the cloud using Arduino or Raspberry Pi.
8. Write a program on Arduino or Raspberry Pi to retrieve temperature and humidity data from the Cloud.
9. Write a program to create TCP Server on Arduino and Respond with humidity data to TCP Client when requested.
10. Write a program to create UDP Server on Arduino and Respond with humidity data to UDP Client when requested.

**Reference:**

1. **IOT and Cloud Computing Lab** Manual, Department of CSE, CMRIT, Hyd.

**Micro-Projects:** Student must submit a report on one of the following Micro–Projects before commencement of second internal examination.

1. Air Pollution Meter
2. Smart Garbage Collector
3. IoT Based Smart street light
4. Humidity & Temperature Monitoring
5. Baggage Tracker
6. IoT based Gas leakage Monitoring
7. Circuit Breakage Detection
8. IoT based Smart irrigation system
9. Anti-Theft Flooring System
10. IoT based water level Monitoring system

# WEEK-1
# Arduino Installation

1. **Aim:** Install necessary software for Arduino and Raspberry Pi.

**Arduino:**

- Arduino is a platform that makes it easy for you to build projects using electronics.
- IoT is a way of using electronics - to make electronic modules talk to each other remotely and wirelessly (often using a Cloud) to solve problems.
- Now, Arduino can also help you easily build IoT projects in two ways: Using traditional Arduino boards and attaching communication breakout modules (like nRF Bluetooth, WiFi, LoRA, GSM, etc) to them.

- Arduino is a micro controller that can be connected to one or more sensors and help you capture the data or information and then pass it on to processor. If you know the full stack of IoT then you should also look at Raspberry.

- RaspPi is a microprocessor so the basic difference between Arduino and RasPi is that

  RaspPi is controller plus processor and Arduino is just a micro controller.

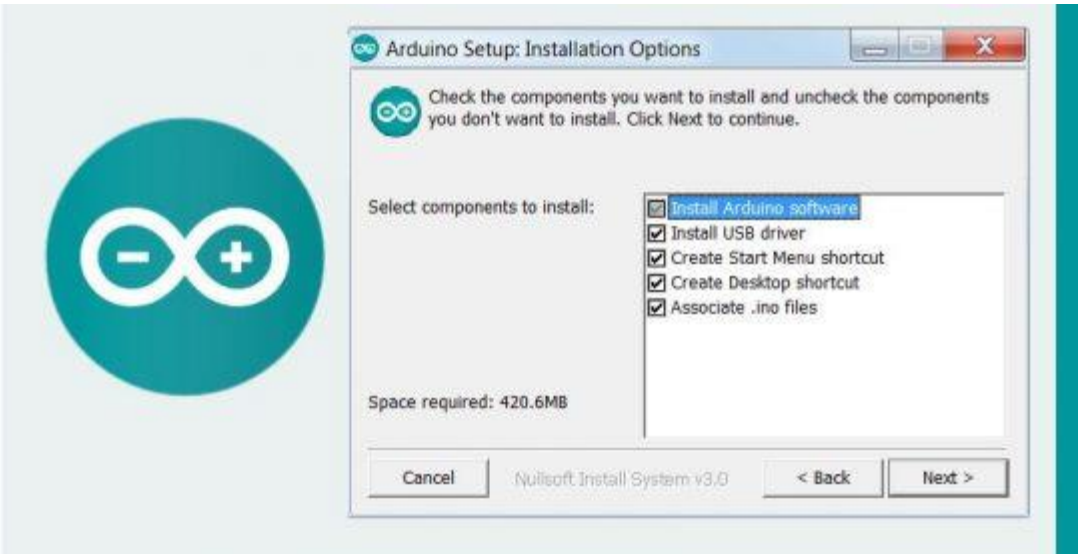
**ARDUINO:**

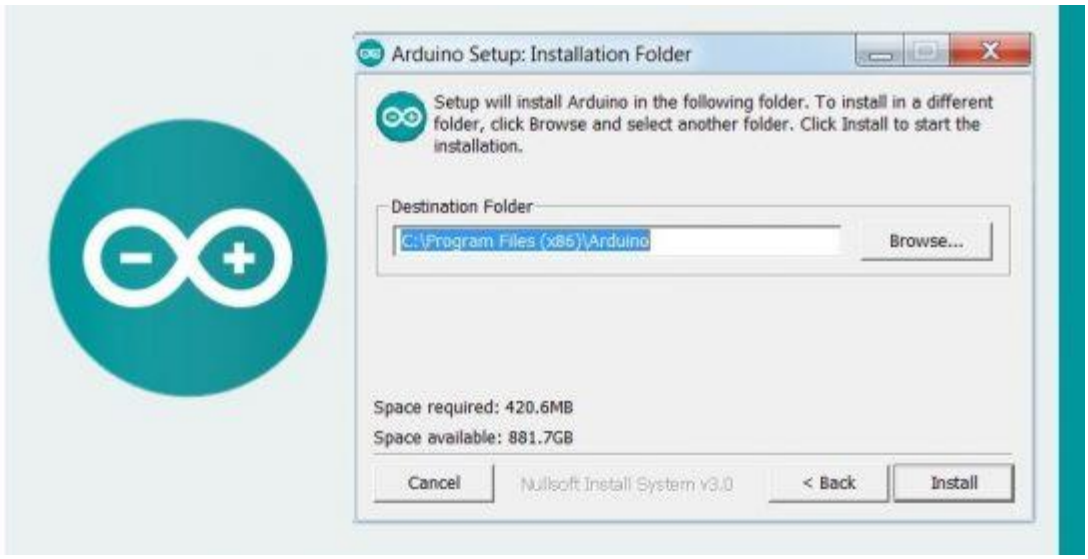
- Go to the Arduino website and click the download link to go to the download page.
- After downloading, locate the downloaded file on the computer and extract the folder from the downloaded zipped file. Copy the folder to a suitable place such as your desktop.






Read the Arduino License agreement and click the “I Agree” button.

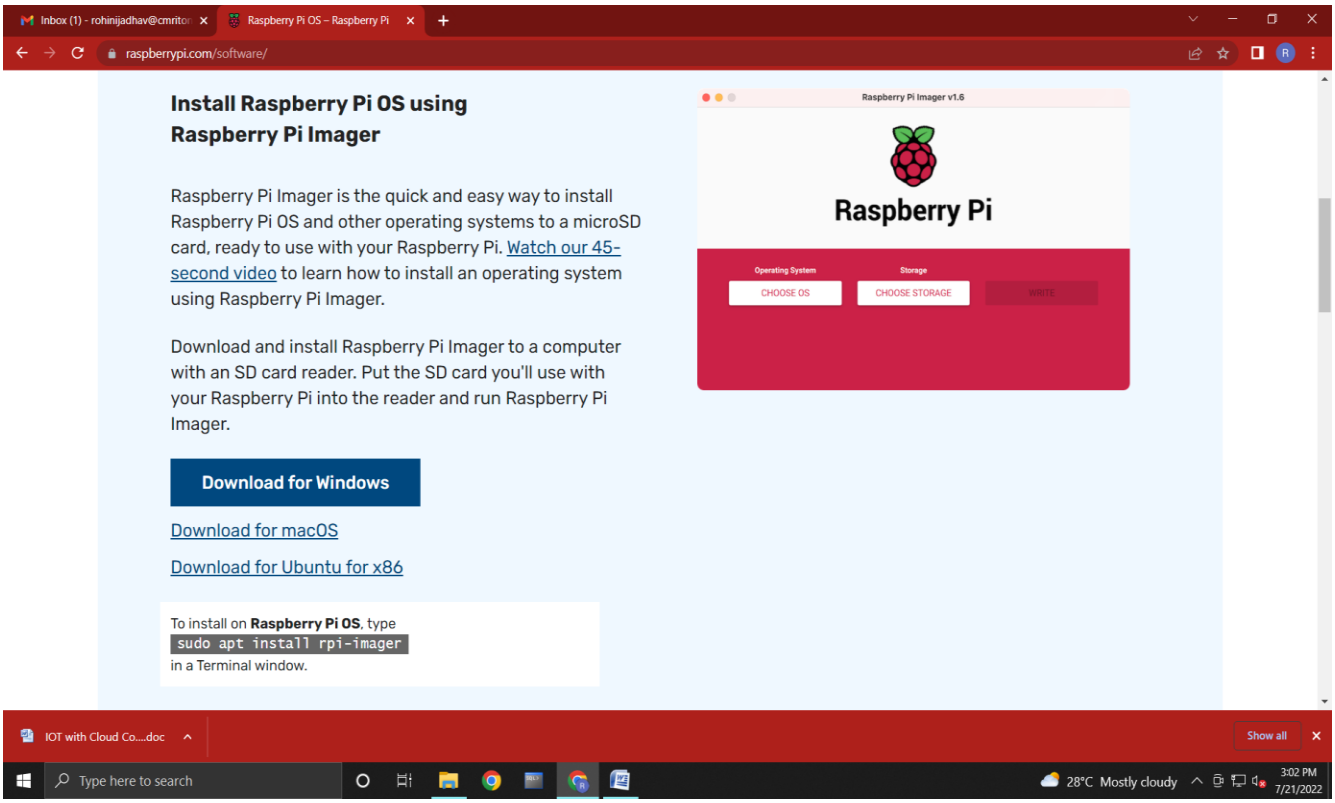


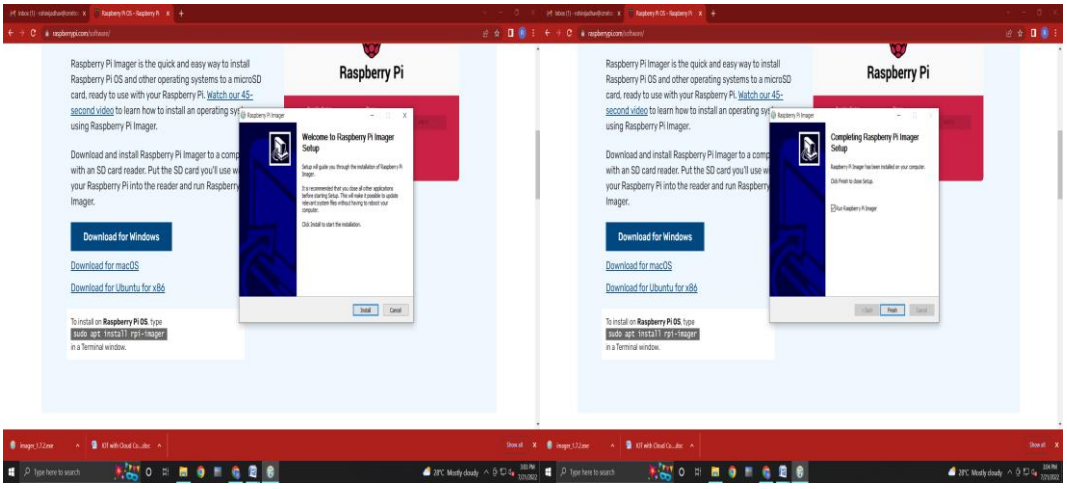


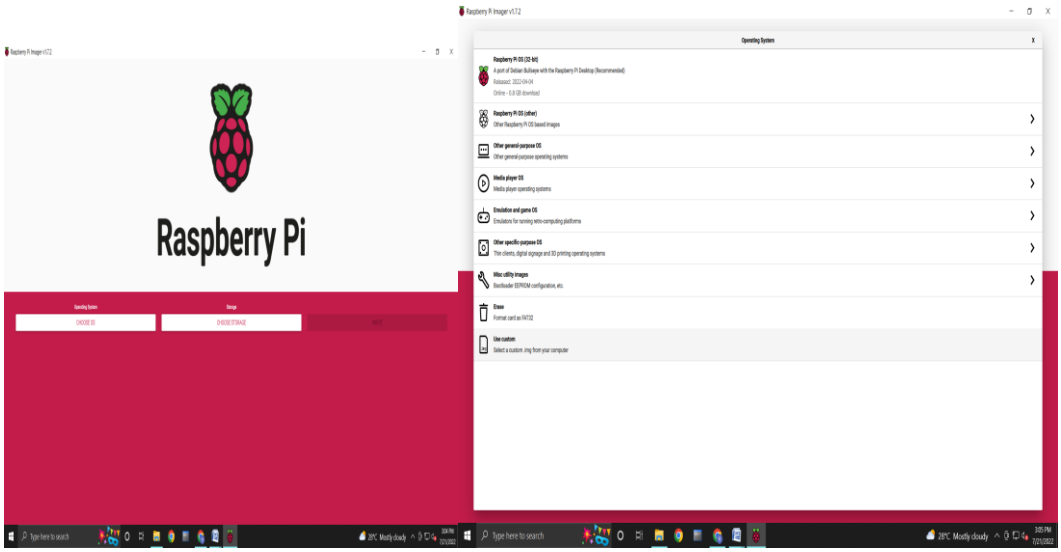

The Arduino software will start to install.

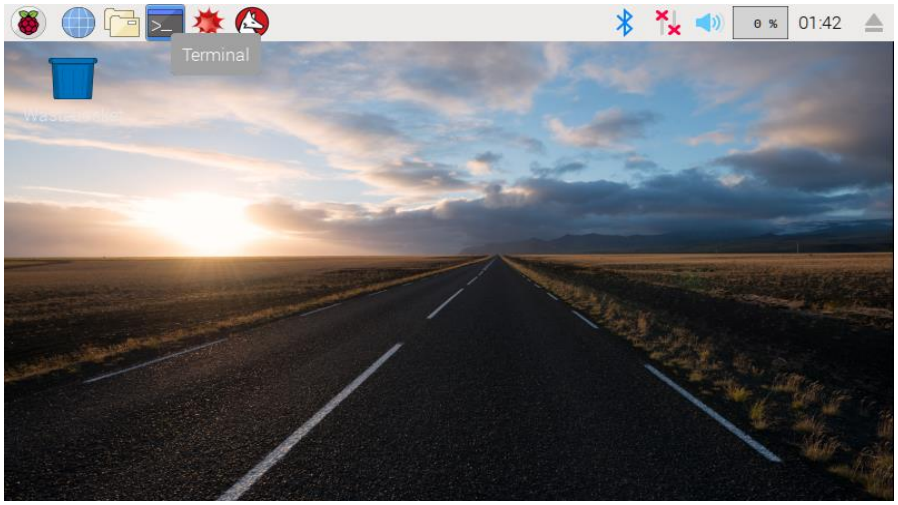


# Raspberry Pi



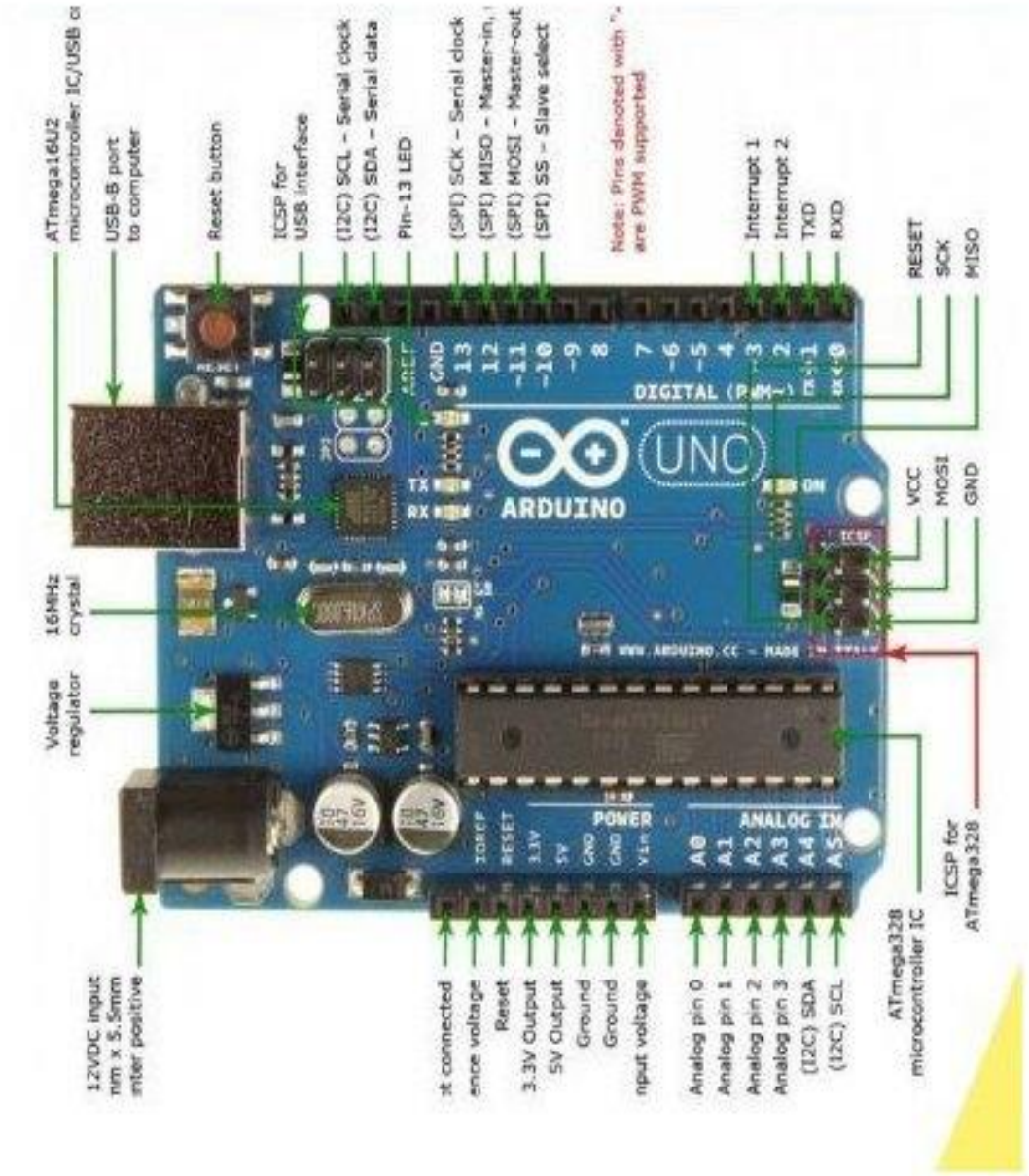



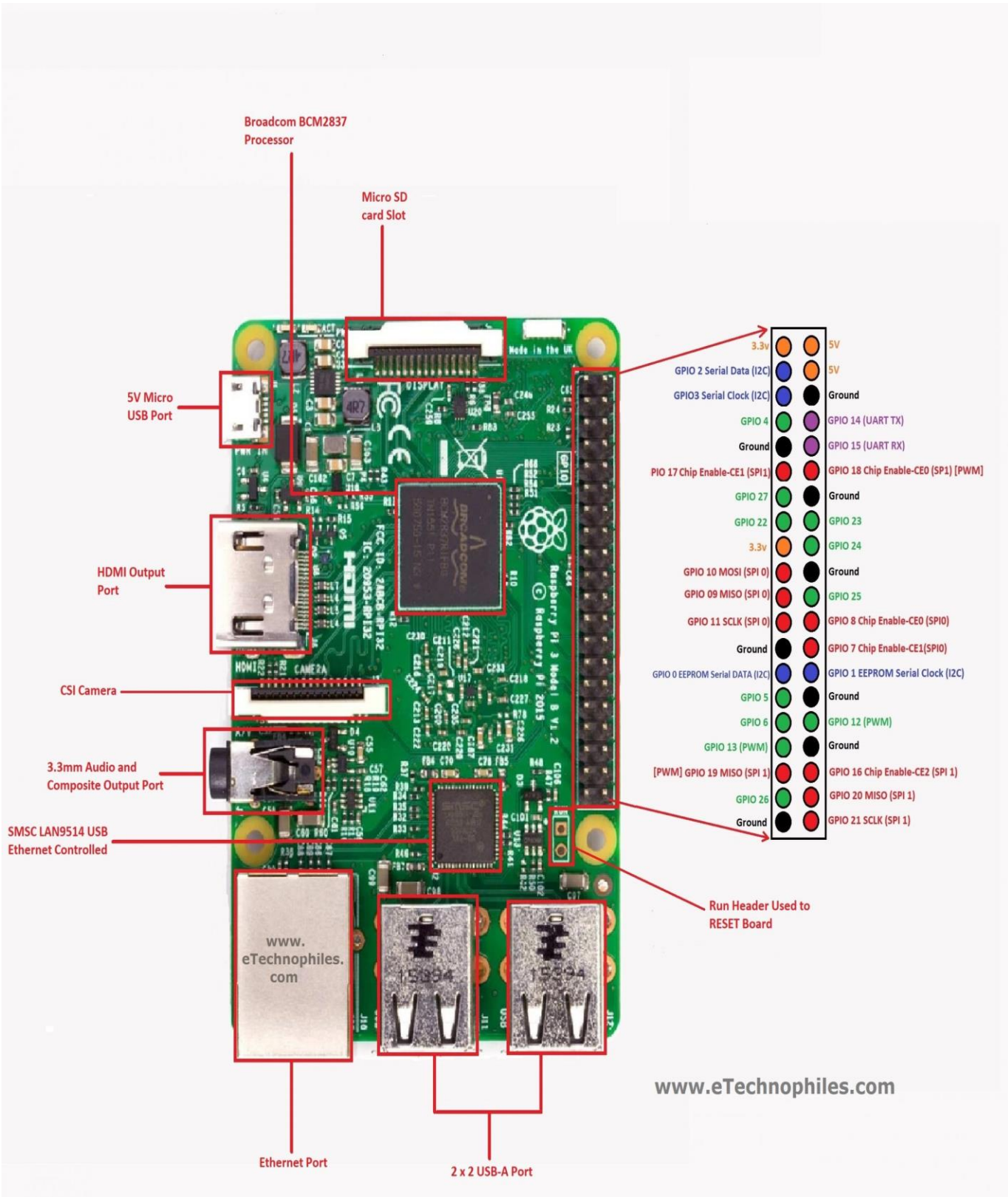

Raspberry Pi
Operating System


Terminal
0 %
01:42

## WEEK-2

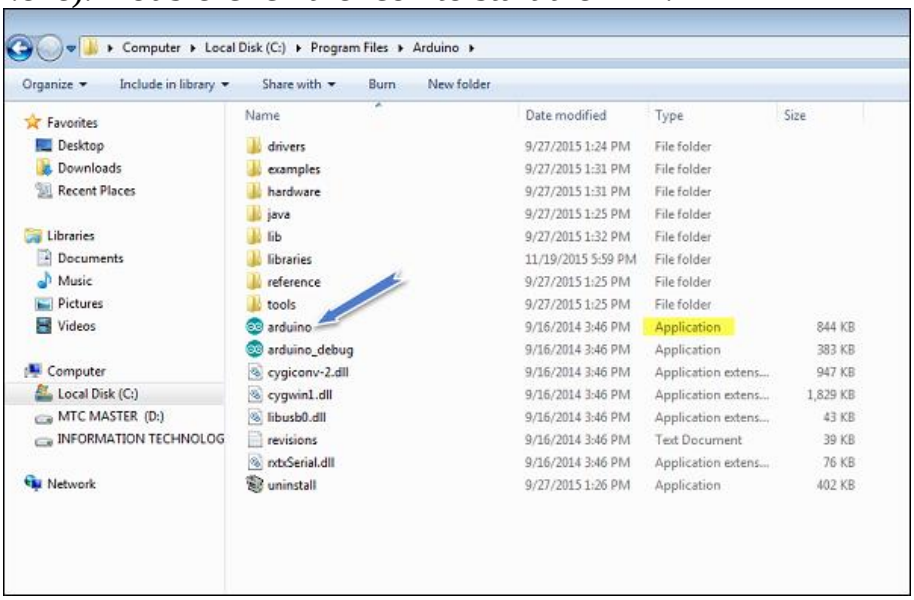
**AIM:** Familiarization with Arduino and Raspberry Pi boards.




Broadcom BCM2837 Processor
Micro SD card Slot
5V Micro USB Port
HDMI Output Port
CSI Camera
3.3mm Audio and Composite Output Port
SMSC LAN9514 USB Ethernet Controlled
Run Header Used to RESET Board
Ethernet Port
2 x 2 USB-A Port
www.eTechnophiles.com
3.3v
5V
GPIO 2 Serial Data (I2C)
5V
GPIO3 Serial Clock (I2C)
Ground
GPIO 4
GPIO 14 (UART TX)
Ground
GPIO 15 (UART RX)
PIO 17 Chip Enable-CE1 (SPI1)
GPIO 18 Chip Enable-CE0 (SP1) [PWM]
GPIO 27
Ground
GPIO 22
GPIO 23
3.3v
GPIO 24
GPIO 10 MOSI (SPI 0)
Ground
GPIO 09 MISO (SPI 0)
GPIO 25
GPIO 11 SCLK (SPI 0)
GPIO 8 Chip Enable-CE0 (SPI0)
Ground
GPIO 7 Chip Enable-CE1(SPI0)
GPIO 0 EEPROM Serial DATA (I2C)
GPIO 1 EEPROM Serial Clock (I2C)
GPIO 5
Ground
GPIO 6
GPIO 12 (PWM)
GPIO 13 (PWM)
Ground
[PWM] GPIO 19 MISO (SPI 1)
GPIO 16 Chip Enable-CE2 (SPI 1)
GPIO 26
GPIO 20 MISO (SPI 1)
Ground
GPIO 21 SCLK (SPI 1)

***Launch Arduino IDE.***

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.



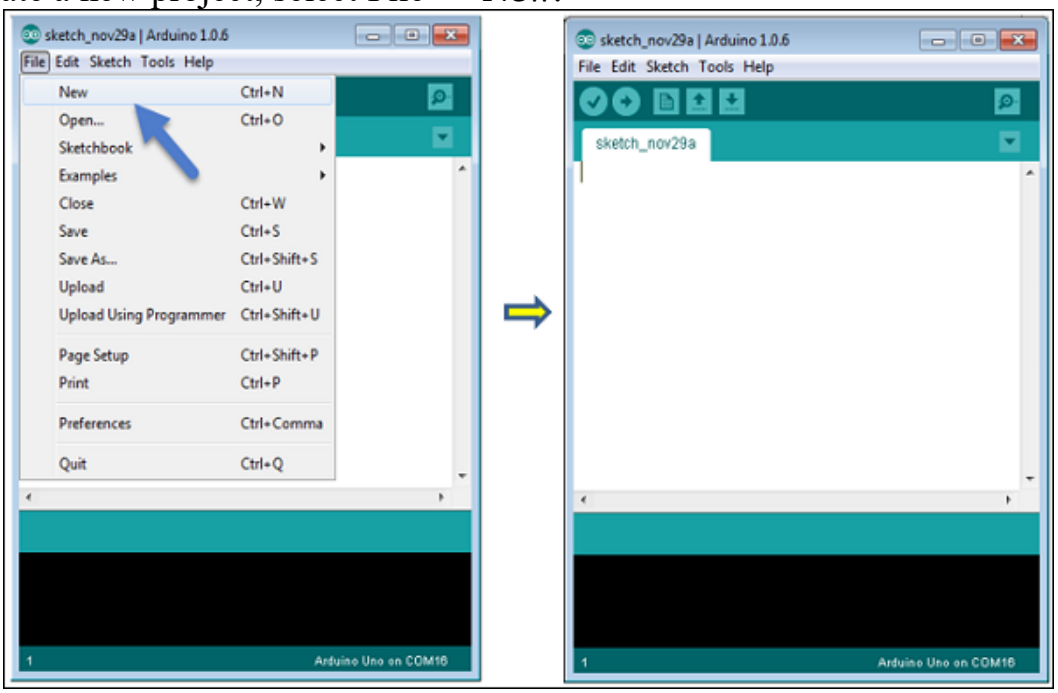

***Step 5 − Open your first project.***

Once the software starts, you have two options −

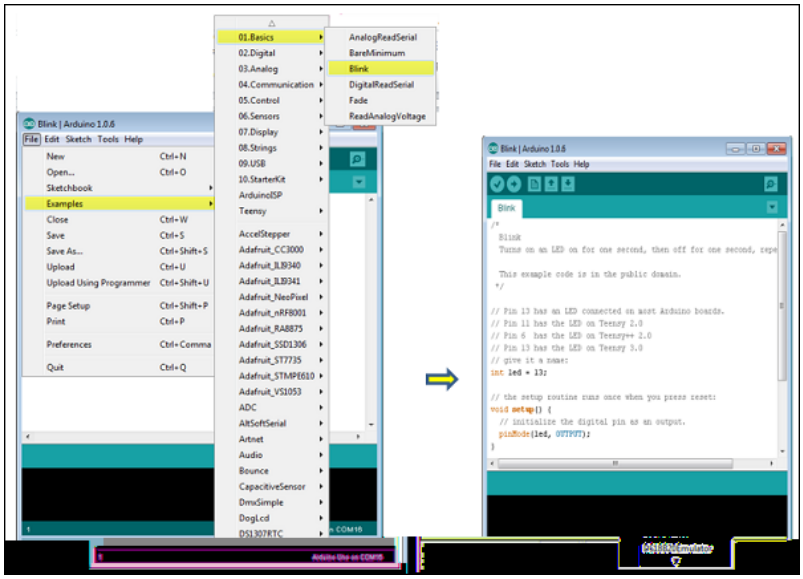
Create a new project.

Open an existing project example.

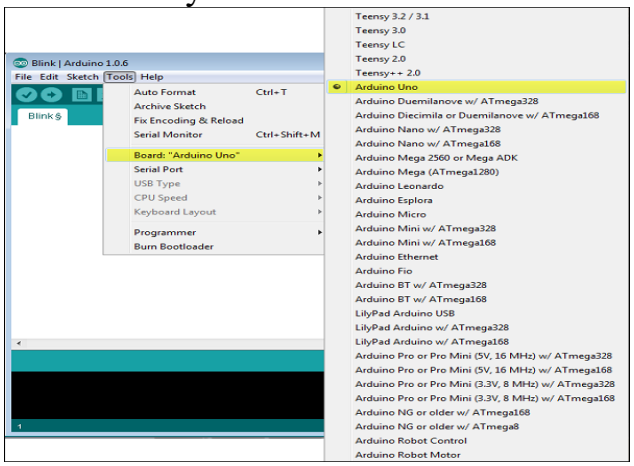
To create a new project, select File → **New**.



To open an existing project example,
select File
→ Example
→ Basics
→ Blink.
Here, we are selecting just one of the examples with the name **Blink**. It turns the LED on and off with some time delay. You can select any other example from the list.



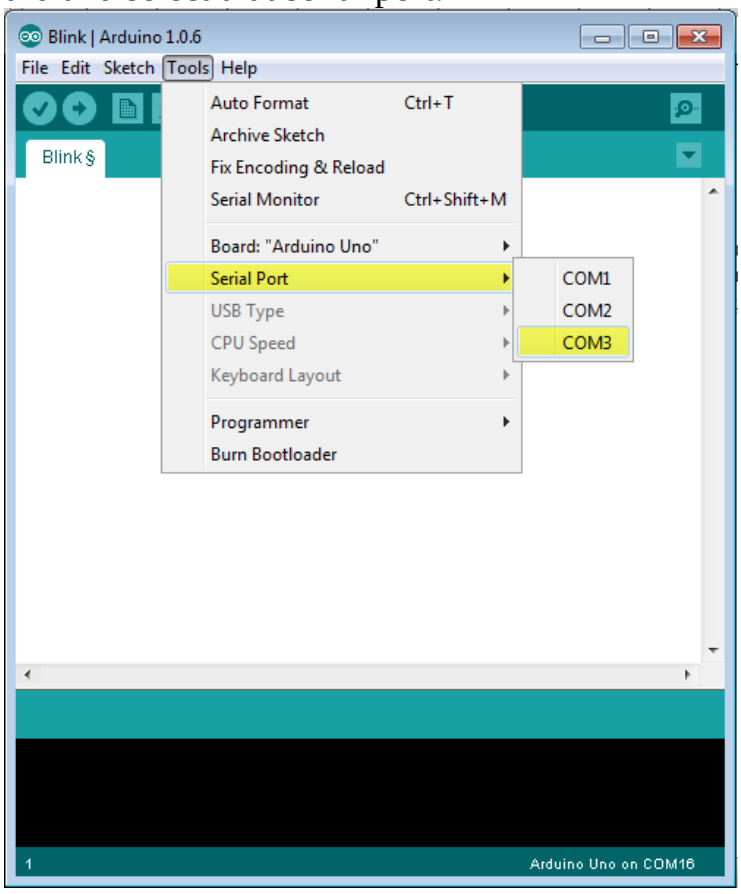

***Step 6 − Select your Arduino board.***
To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.
Go to Tools → Board and select your board.



Here, we have selected Arduino Uno board according to our tutorial, but you must select the name matching the board that you are using.

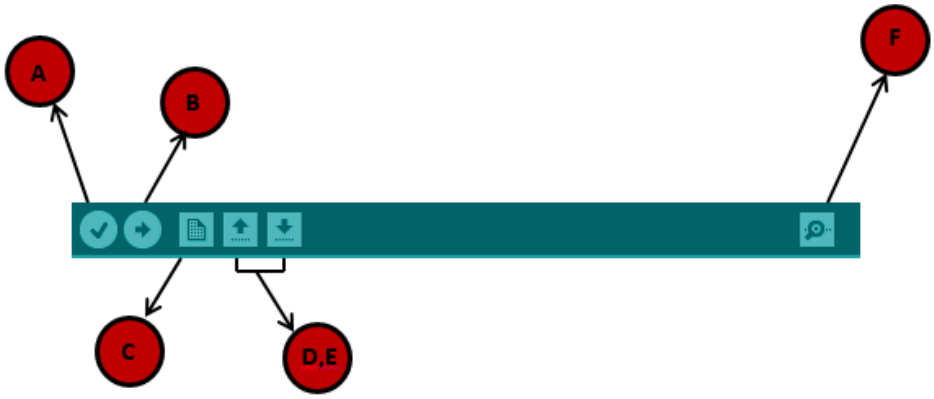
***Step 7 − Select your serial port.***

Select the serial device of the Arduino board. Go to **Tools → Serial Port** menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.




***Step 8 − Upload the program to your board.***

Before explaining how we can upload our program to the board, we must demonstrate the function of each symbol appearing in the Arduino IDE toolbar.




**A** − Used to check if there is any compilation error.
**B** − Used to upload a program to the Arduino board.
**C** − Shortcut used to create a new sketch.
**D** − Used to directly open one of the example sketch.
**E** − Used to save your sketch.
**F** − Serial monitor used to receive serial data from the board and send the serial data to the board.

Now, simply click the "Upload" button in the environment. Wait a few seconds; you will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

**Note** − If you have an Arduino Mini, NG, or other board, you need to press the reset button physically on the board, immediately before clicking the upload button on the Arduino Software.

Arduino - Program Structure

In this chapter, we will study in depth, the Arduino program structure and we will learn more new terminologies used in the Arduino world. The Arduino software is open-source. The source code for the Java environment is released under the GPL and the C/C++ microcontroller libraries are under the LGPL.

**Sketch** − The first new terminology is the Arduino program called "**sketch**".

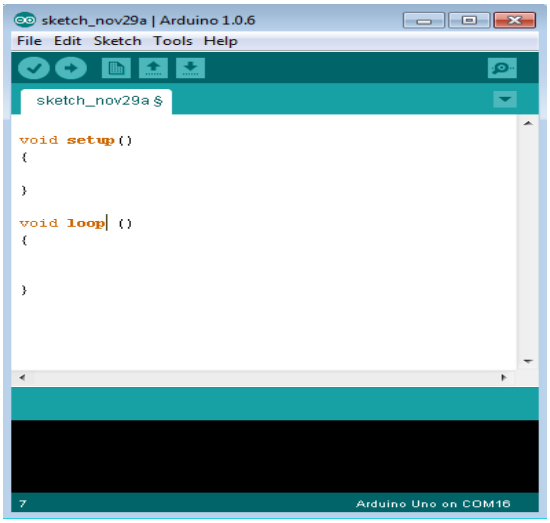
Structure

Arduino programs can be divided in three main parts: **Structure, Values** (variables and constants), and **Functions**. In this tutorial, we will learn about the Arduino software program, step by step, and how we can write the program without any syntax or compilation error.

Let us start with the **Structure**. Software structure consist of two main functions −

Setup( ) function

Loop( ) function




Void setup ( ) {

}

**PURPOSE** − The **setup()** function is called when a sketch starts. Use it to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the Arduino board.

**INPUT** − -

**OUTPUT** − -

**RETURN** − -

Void Loop ( ) {

}

**PURPOSE** − After creating a **setup()** function, which initializes and sets the initial values, the **loop()** function does precisely what its name suggests, and loops consecutively, allowing your program to change and respond. Use it to actively control the Arduino board.

**INPUT** − -

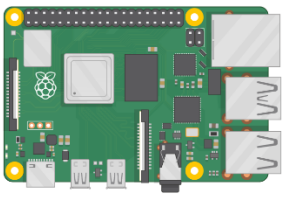
**OUTPUT** − -

**RETURN** − -

***RASPBERRY PI***

There are several models of Raspberry pi and for most people Raspberry Pi 3 Model B is the one to choose. Raspberry Pi 3 Model B is the newest, fastest, and easiest to use.

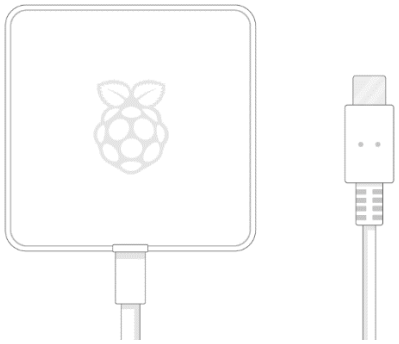
Raspberry Pi 3 comes with either 2GB or 4GB of RAM. For most educational purposes and hobbyist projects, and for use as a desktop computer, 2GB is enough.



Raspberry Pi Zero and Zero W are smaller and require less power, so they're useful for portable projects such as robots. It's generally easier to start a project with Raspberry Pi 3 , and to move to Pi Zero when you have a working prototype that a smaller Pi would be useful for.

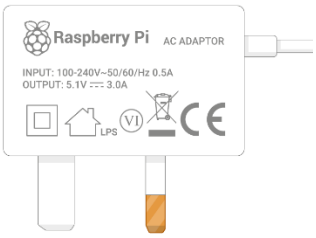
***A power supply***

To connect to a power socket, all Raspberry Pi models have a USB port (the same found on many mobile phones): either USB-C for Raspberry Pi 3 , or micro USB for Raspberry Pi 3, 2 and 1.



You need a power supply that provides:

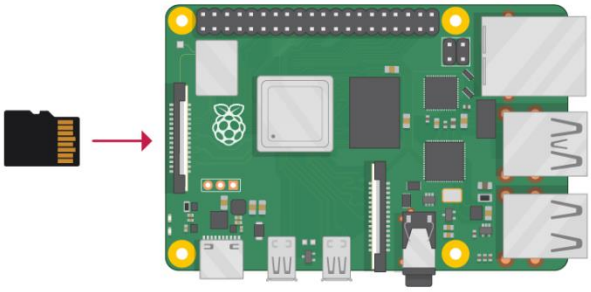
At least 3.0 amps for Raspberry Pi 3




At least 2.5 amps for Raspberry Pi 3



- We recommend using our official Universal Power Supply.
- A microSD card
- Your Raspberry Pi needs an SD card to store all its files and the Raspbian operating system.



- You need a microSD card with a capacity of at least 8 GB.
- Many sellers supply SD cards for Raspberry Pi that are already set up with Raspbian and ready to go.

***A keyboard and a mouse***

To start using your Raspberry Pi, you need a USB keyboard and a USB mouse. Once you've set your Pi up, you can use a Bluetooth keyboard and mouse, but you'll need a USB keyboard and mouse for the first setup.

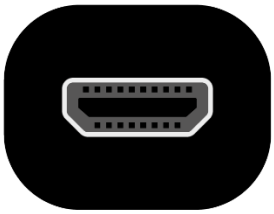
***A TV or computer screen***

To view the Raspbian desktop environment, you need a screen, and a cable to link the screen and the Pi. The screen can be a TV or a computer monitor. If the screen has built-in speakers, the Pi is able to use these to play sound.

***HDMI***

The Raspberry Pi has a HDMI output port that is compatible with the HDMI port of most modern TVs and computer monitors. Many computer monitors may also have DVI or VGA ports.

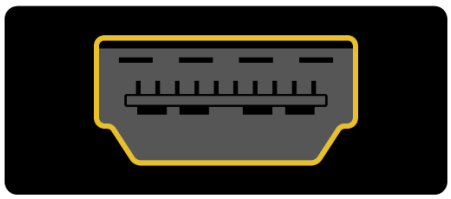
Raspberry Pi 3 has two micro HDMI ports, allowing you to connect two separate monitors.



You need either a micro HDMI-to-HDMI cable, or a standard HDMI-to-HDMI cable plus a micro HDMI-to-HDMI adapter, to connect Raspberry Pi 3 to a screen.

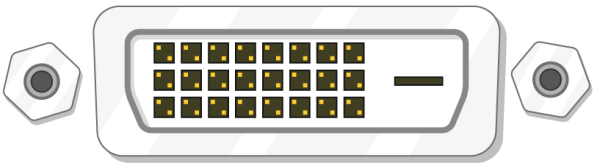


Raspberry Pi 1, 2 and 3 have a single full-size HDMI port, so you can connect them to a screen using a standard HDMI-to-HDMI cable.



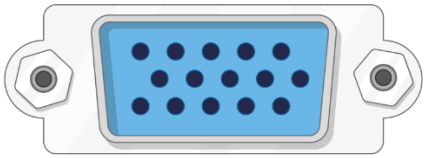
**DVI**

If your screen has a DVI port, you can connect the Pi to it using a HDMI-to-DVI cable.

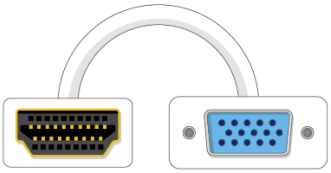


**VGA**

Some screens only have a VGA port.



To connect your Pi to such a screen, you can use a HDMI-to-VGA adapter.



***Headphones or speakers***

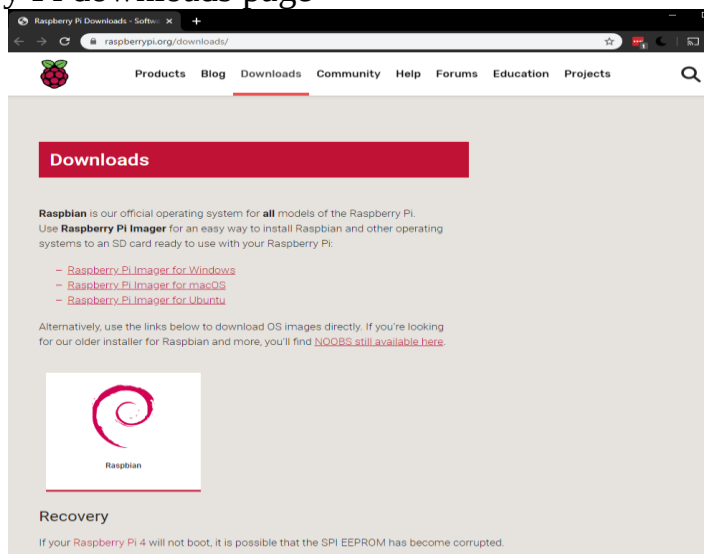
The large Raspberry Pi models (but not Pi Zero/Zero W) have a standard audio port like the one on your smart phone or MP3 player. If you want to, you can connect your headphones or speakers so that your Raspberry Pi can play sound. If the screen you're connecting your Raspberry Pi to has built-in speakers, Raspberry Pi can play sound through these.

***An Ethernet cable***

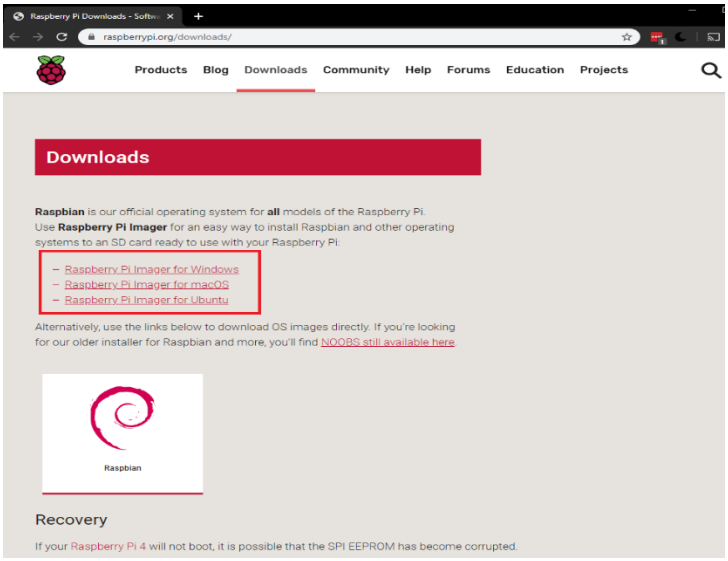
The large Raspberry Pi models (but not Pi Zero/Zero W) have a standard Ethernet port to connect them to the internet; to connect Pi Zero to the internet, you need a USB-to-Ethernet adaptor.
Raspberry Pi 3 , 3, and Pi Zero W can also be wirelessly connected to the internet.

***Set up your SD card***

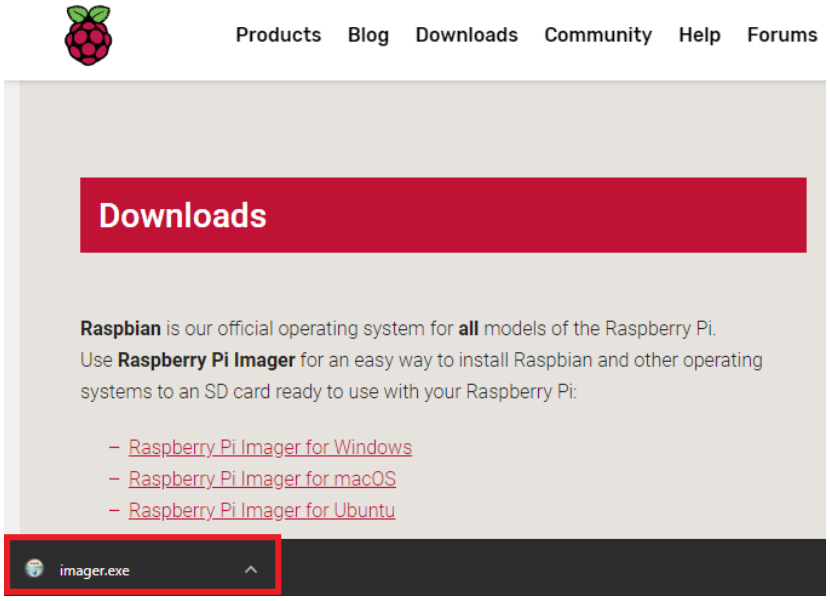
If you have an SD card that doesn't have the Raspbian operating system on it yet, or if you want to reset your Raspberry Pi, you can easily install Raspbian yourself. To do so, you need a computer that has an SD card port — most laptop and desktop computers have one.
The Raspbian operating system via the Raspberry Pi Imager
Using the Raspberry Pi Imager is the easiest way to install Raspbian on your SD card.
Note: More advanced users looking to install a particular operating system should use this guide to installing operating system images.
Download and launch the Raspberry Pi Imager
Visit the Raspberry Pi downloads page




Click on the link for the Raspberry Pi Imager that matches your operating system



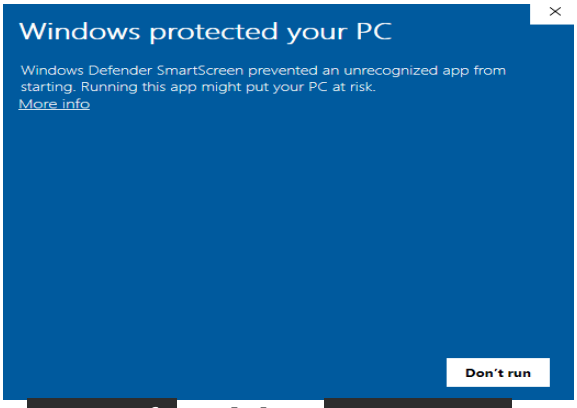

When the download finishes, click it to launch the installer



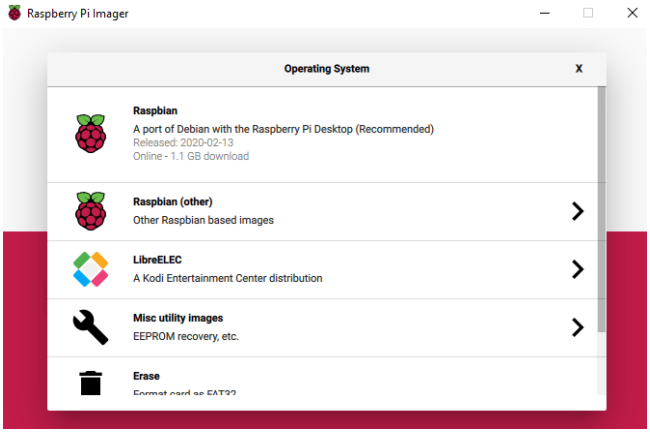

Using the Raspberry Pi Imager

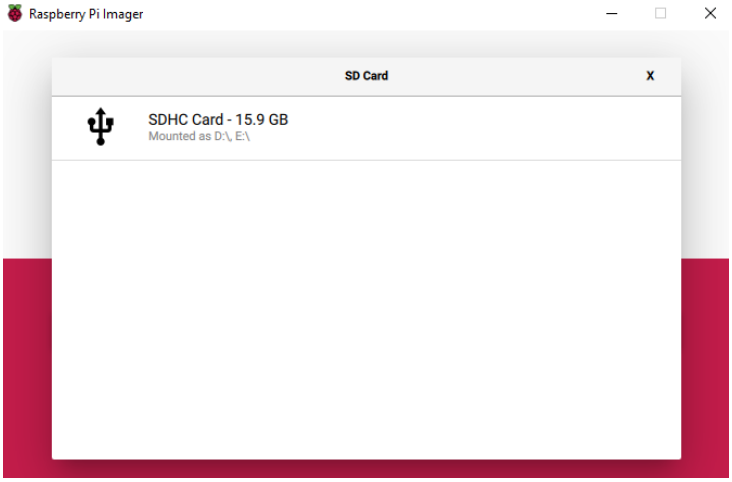
Anything that's stored on the SD card will be overwritten during formatting. If your SD card currently has any files on it, e.g. from an older version of Raspbian, you may wish to back up these files first to prevent you from permanently losing them.

When you launch the installer, your operating system may try to block you from running it. For example, on Windows I recieve the following message:



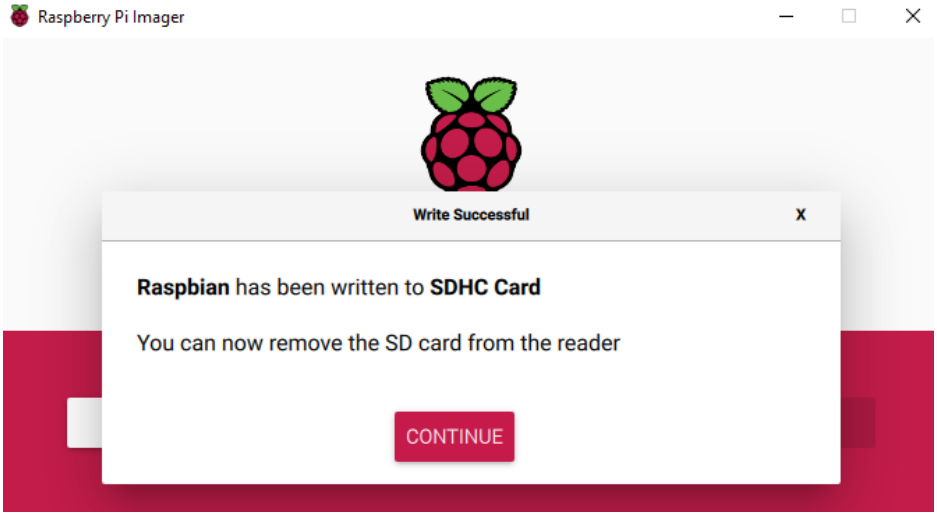

If this pops up, click on More info and then Run anyway
Follow the instructions to install and run the Raspberry Pi Imager
Insert your SD card into the computer or laptop SD card slot
In the Raspberry Pi Imager, select the OS that you want to install and the SD card you would like to install it on



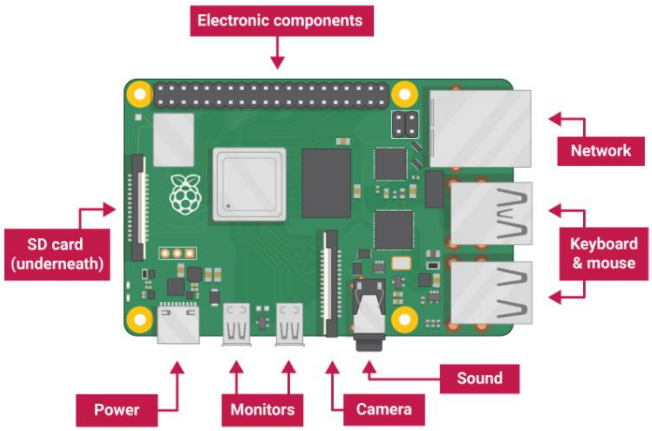



Then simply click the WRITE button
Wait for the Raspberry Pi Imager to finish writing
Once you get the following message, you can eject your SD card

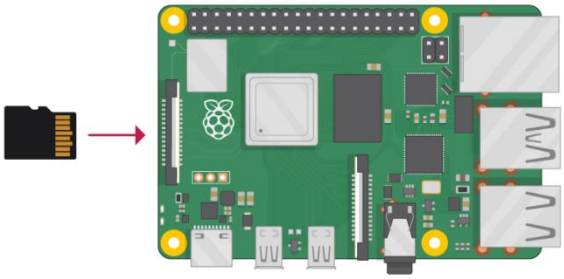



**Connect your Raspberry Pi**

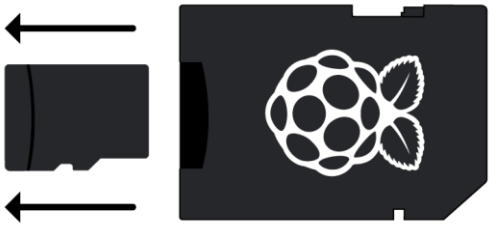
Now get everything connected to your Raspberry Pi. It's important to do this in the right order, so that all your components are safe.



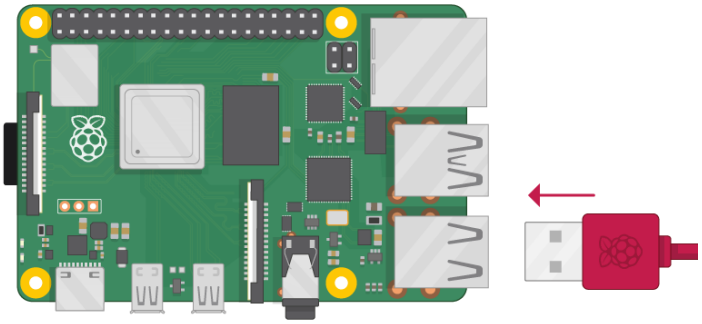

Insert the SD card you've set up with Raspbian (via NOOBS) into the microSD card slot on the underside of your Raspberry Pi.



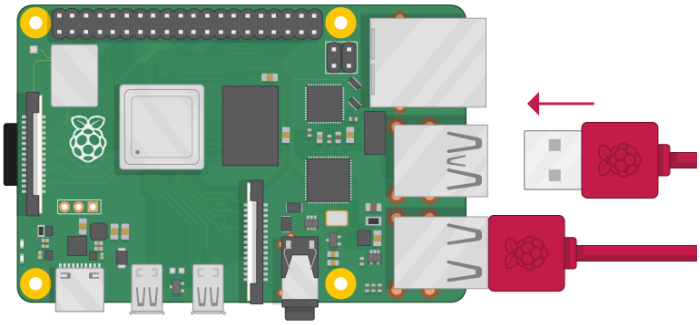
Note: Many microSD cards come inside a larger adapter — you can slide the smaller card out using the lip at the bottom.



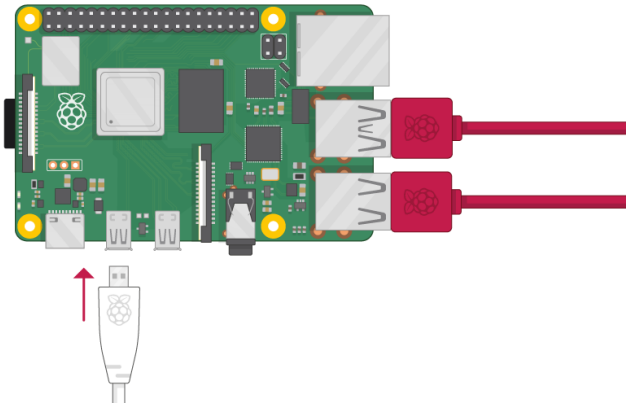
Find the USB connector end of your mouse's cable, and connect the mouse to a USB port on Raspberry Pi (it doesn't matter which port you use).



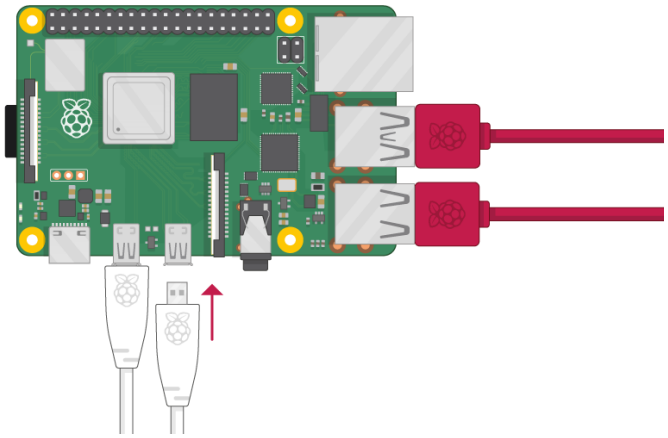
Connect the keyboard in the same way.



Make sure your screen is plugged into a wall socket and switched on.
Look at the HDMI port(s) on the Raspberry Pi — notice that they have a flat side on top.
Use a cable to connect the screen to Raspberry Pi's HDMI port — use an adapter if necessary.
Raspberry Pi 3
Connect your screen to the first of Raspberry Pi 3 's HDMI ports, labelled HDMI0.

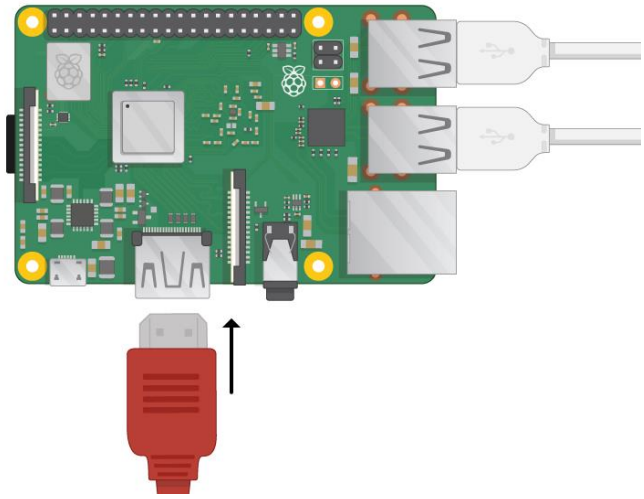


You can connect an optional second screen in the same way.



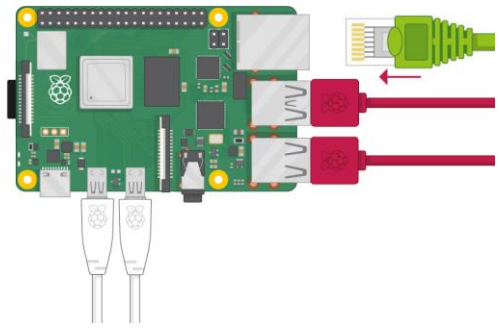
Raspberry Pi 1, 2, 3

Connect your screen to the single HDMI port.

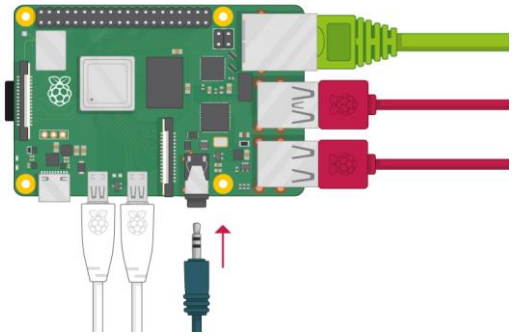


Note: nothing will display on the screen, because your Raspberry Pi is not running yet.

If you want to connect your Raspberry Pi to the internet via Ethernet, use an Ethernet cable to connect the Ethernet port on Raspberry Pi to an Ethernet socket on the wall or on your internet router. You don't need to do this if you want to use wireless connectivity, or if you don't want to connect to the internet.

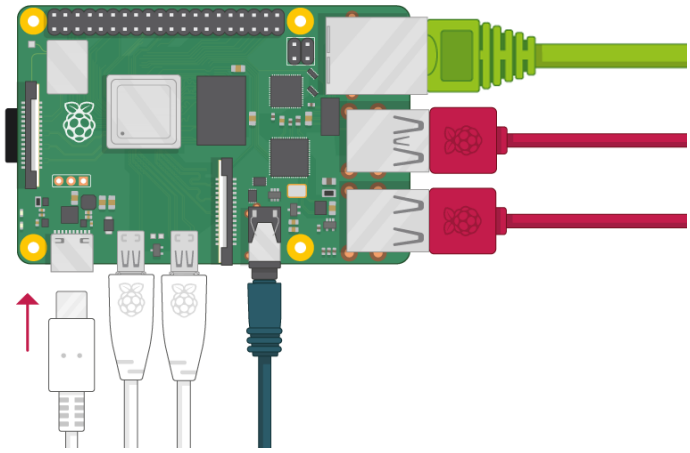


If the screen you are using has speakers, sound will play through those. Alternatively, connect headphones or speakers to the audio port if you prefer.

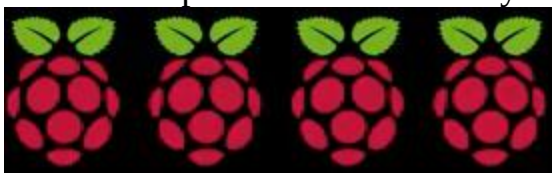


Start up your Raspberry Pi

Your Raspberry Pi doesn't have a power switch: as soon as you connect it to a power outlet, it will turn on.
Plug the USB power supply into a socket and connect it to your Raspberry Pi's power port.



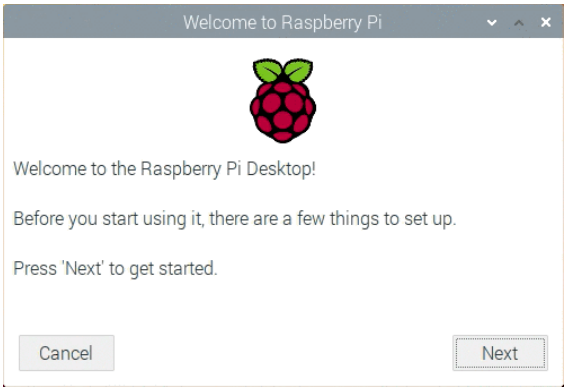
You should see a red LED light up on the Raspberry Pi, which indicates that Raspberry Pi is connected to power. As it starts up (this is also called booting), you will see raspberries appear in the top left-hand corner of your screen.



First-time startup with NOOBS
After a few seconds the Raspbian Desktop will appear.



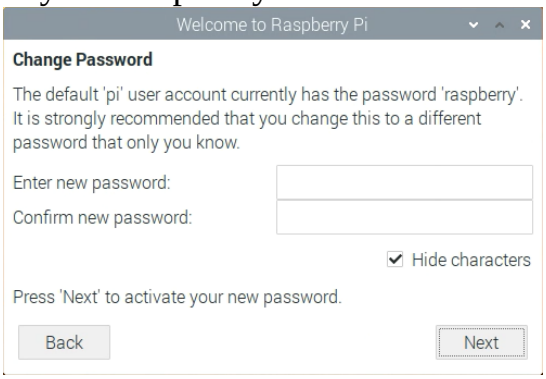

Finish the setup
When you start your Raspberry Pi for the first time, the Welcome to Raspberry Pi application will pop up and guide you through the initial setup.



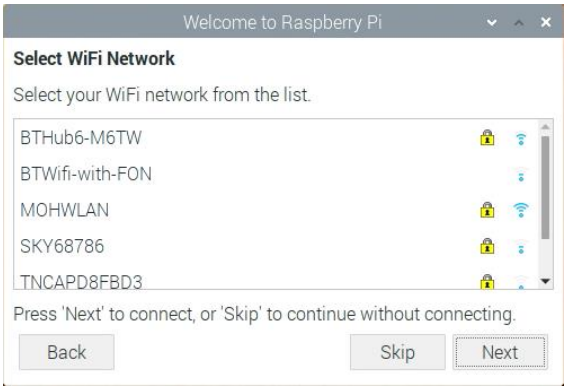

Click Next to start the setup.
Set your Country, Language, and Timezone, then click Next again.




Enter a new password for your Raspberry Pi and click Next.

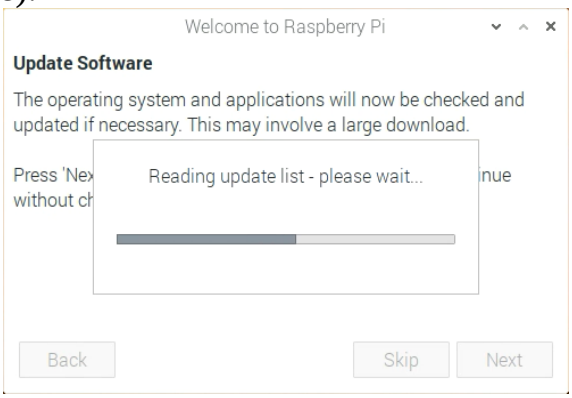



Connect to your WiFi network by selecting its name, entering the password, and clicking Next.



Note: if your Raspberry Pi model doesn't have wireless connectivity, you won't see this screen.

Click Next let the wizard check for updates to Raspbian and install them (this might take a little while).




Click Done or Reboot to finish the setup.

Note: you will only need to reboot if that's necessary to complete an update.



**WEEK-3**

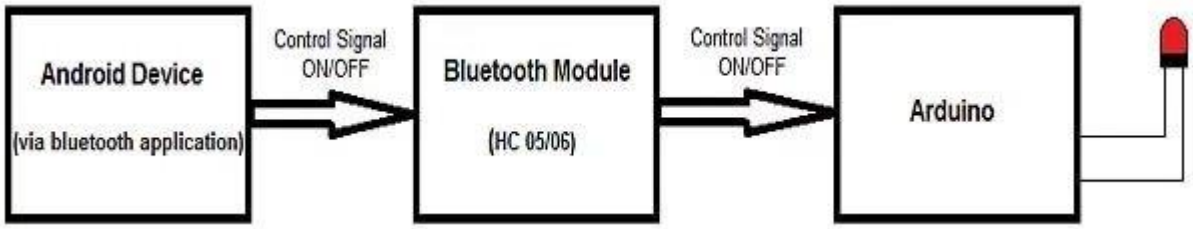
**AIM:** To interface Bluetooth with Arduino/Raspberry Pi and write a program to turn LED ON/OFF when '1'/'0' is received from smart phone using Bluetooth.

**Hardware Requirements**:

1. Arduino UNO
2. Android Smartphone that has Bluetooth.
3. **HC-05** Bluetooth Module
4. Android Studio (To develop the required Android app)
5. USB cable for programming and powering the Arduino

**Procedure:**

There are three main parts to this project. An Android smart phone, a Bluetooth transceiver, and an Arduino.




1. HC 05/06 works on serial communication.
2. The Android app is designed to send serial data to the Arduino Bluetooth module when a button is pressed on the app.
3. The Arduino Bluetooth module at the other end receives the data and sends it to the Arduino through the TX pin of the Bluetooth module (connected to RX pin of Arduino).
4. The code uploaded to the Arduino checks the received data and compares it. If the received data is 1, the LED turns ON.
5. The LED turns OFF when the received data is 0. You can open the serial monitor and watch the received data while connecting.

**SOURCE CODE:**

```
#include <SoftwareSerial.h>

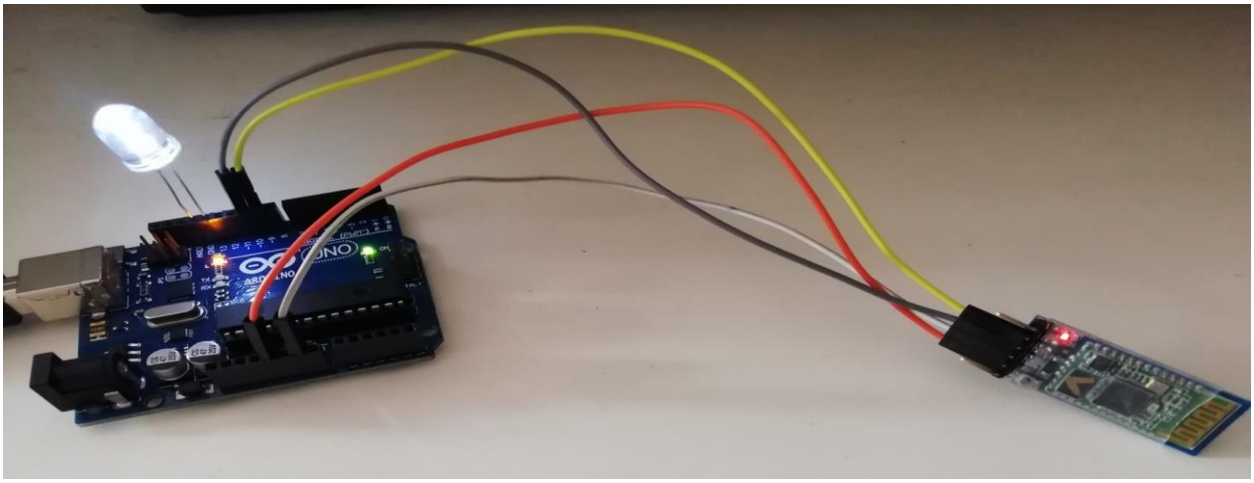
SoftwareSerial Bluetooth(10, 9); // RX, TX

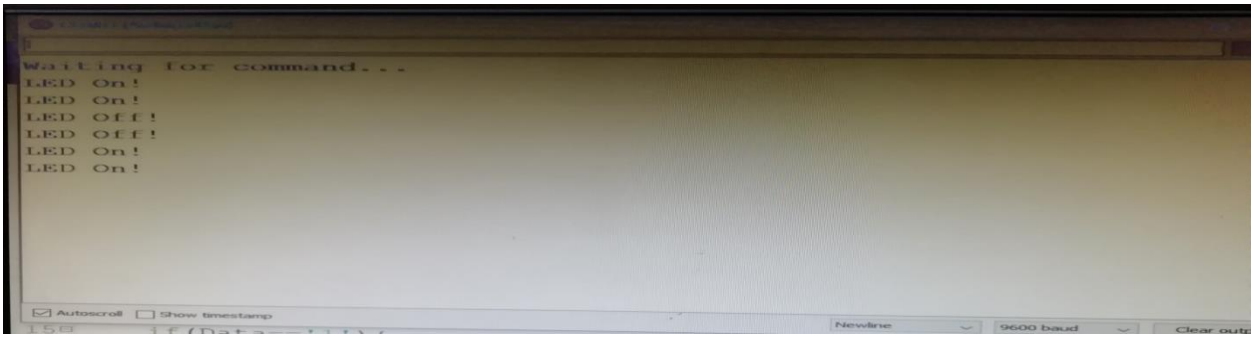
int LED = 13; // the on-board LED

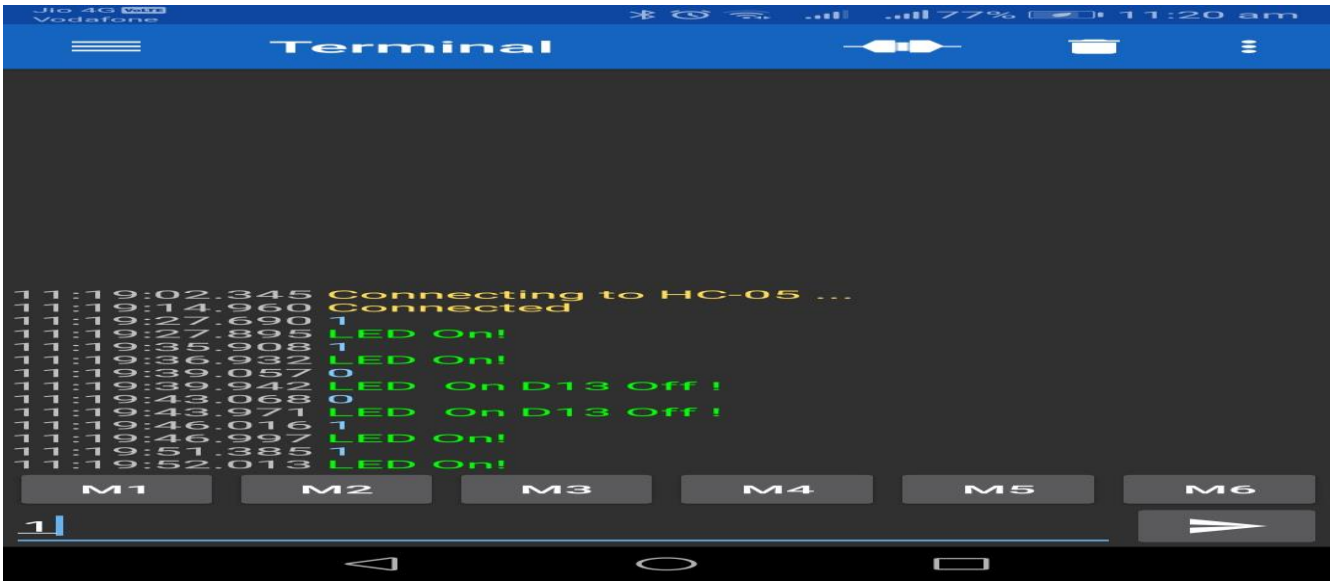
int Data; // the data received
```

```
void setup() {
  Bluetooth.begin(9600);
  Serial.begin(9600);
  Serial.println("Waiting for command...");
  Bluetooth.println("Send 1 to turn on the LED. Send 0 to turn Off");
  pinMode(LED,OUTPUT);
}
void loop() {
  if (Bluetooth.available()){ //wait for data received
  Data=Bluetooth.read();
   if(Data=='1'){
   digitalWrite(LED,1);
   Serial.println("LED On!");
   Bluetooth.println("LED On!");
    }
    else if(Data=='0'){
    digitalWrite(LED,0);
    Serial.println("LED Off!");
     Bluetooth.println("LED  On D13 Off ! ");
    }
    else{;}
  }
delay(100);
}
```

**OUTPUT:**

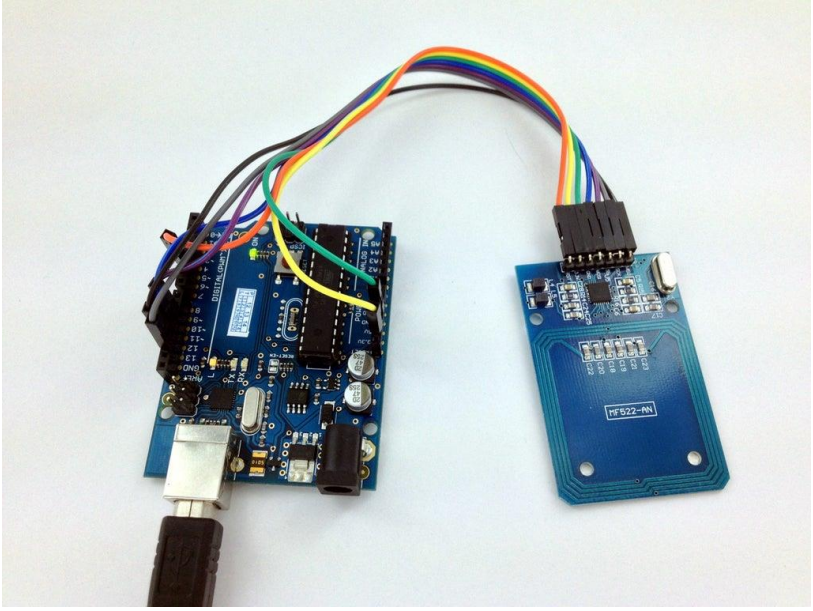



Waiting for command...
LED On!
LED On!
LED Off!
LED Off!
LED On!
LED On!
Autoscroll
Show timestamp
Newline
9600 baud


Jio 4G VoLTE
Vodafone
77%
11:20 am
Terminal
11:19:02.345 Connecting to HC-05 ...
11:19:14.960 Connected
11:19:27.690 1
11:19:27.835 LED On!
11:19:35.908 1
11:19:36.932 LED On!
11:19:39.057 0
11:19:39.942 LED On D13 Off !
11:19:43.068 0
11:19:43.971 LED On D13 Off !
11:19:46.016 1
11:19:46.997 LED On!
11:19:51.385 1
11:19:52.013 LED On!
M1
M2
M3
M4
M5
M6
1

**WEEK-4**

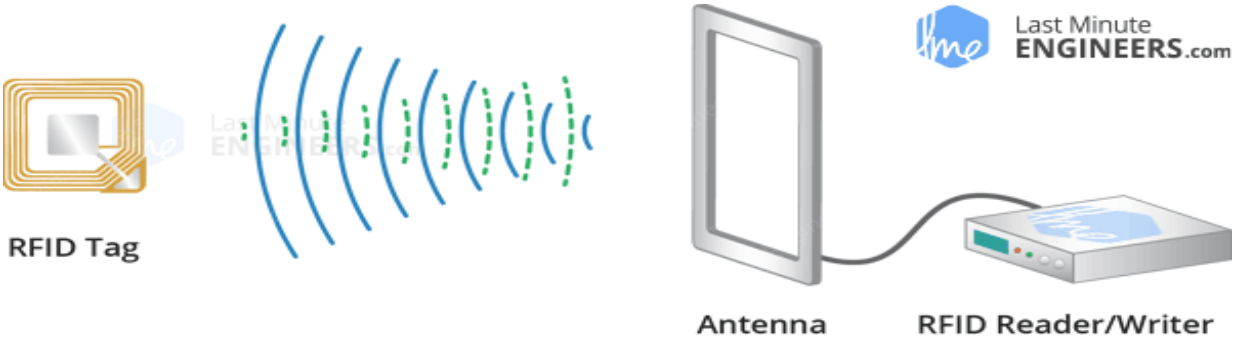
**AIM:** Write a program to implement RFID using Arduino.



# What is RFID technology and how does it work?

An RFID or radio frequency identification system consists of two main components, a tag attached to the object to be identified, and a reader that reads the tag.

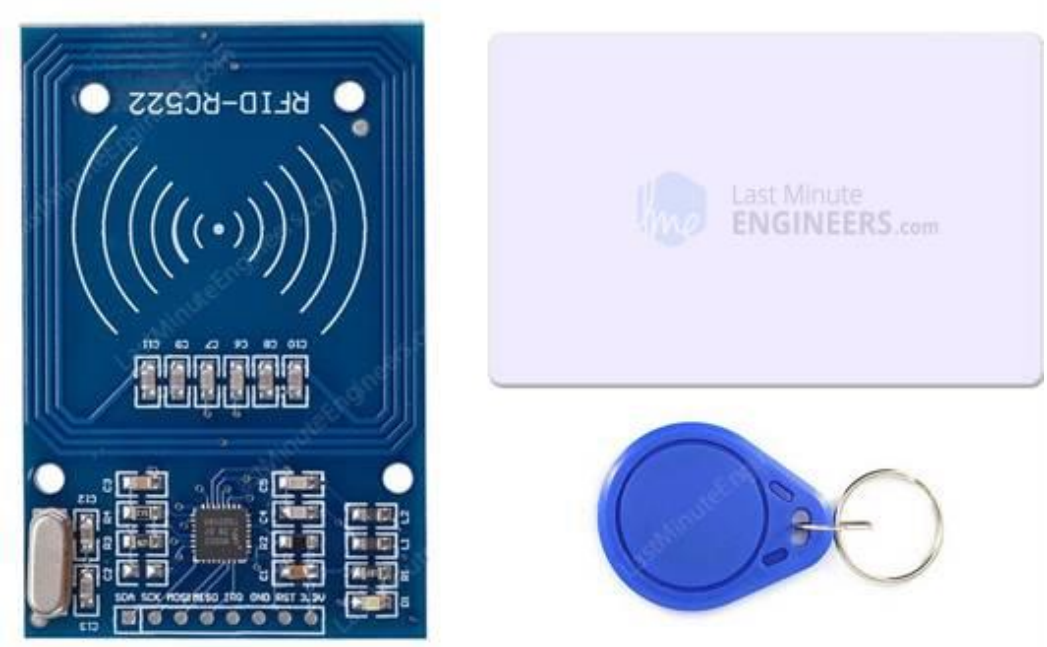
A reader consists of a radio frequency module and an antenna that generates a high frequency electromagnetic field. Whereas the tag is usually a passive device (it does not have a battery). It consists of a microchip that stores and processes information, and an antenna for receiving and transmitting a signal.



When the tag is brought close to the reader, the reader generates an electromagnetic field. This causes electrons to move through the tag's antenna and subsequently powers the chip.

Hardware Overview

The RC522 RFID module based on the MFRC522 IC is one of the cheapest RFID options you can get online for less than four dollars. It usually comes with an RFID card tag and a key fob tag with 1KB of memory. And the best part is that it can write a tag that means you can store any message in it.




The RC522 RFID reader module is designed to create a 13.56MHz electromagnetic field and communicate with RFID tags (ISO 14443A standard tags).

The reader can communicate with a microcontroller over a 4-pin SPI with a maximum data rate of 10 Mbps. It also supports communication over I2C and UART protocols.

The RC522 RFID module can be programmed to generate an interrupt, allowing the module to alert us when a tag approaches it, instead of constantly asking the module "Is there a card nearby?".

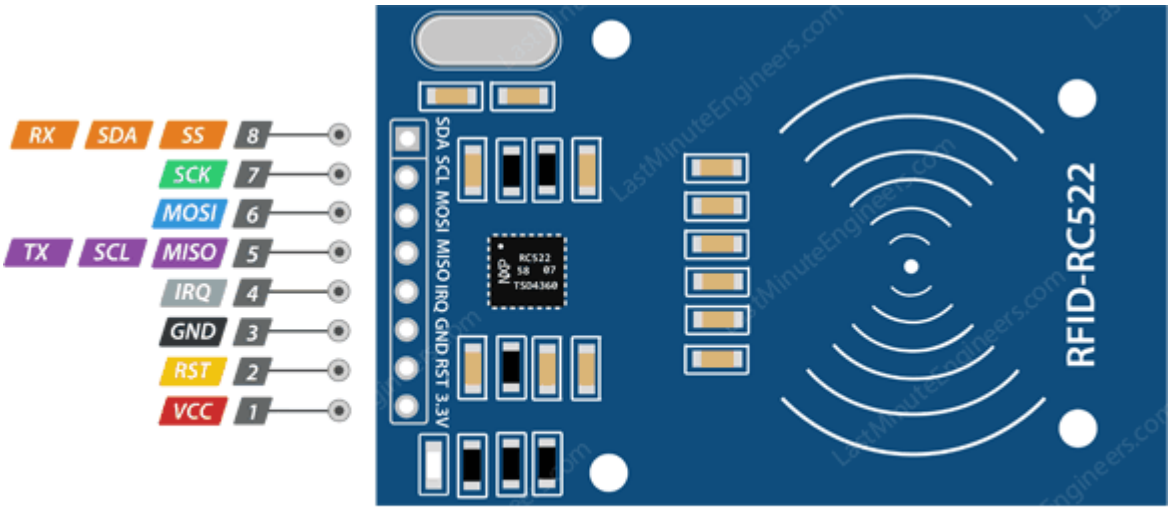
The module's operating voltage ranges from 2.5 to 3.3V, but the good news is that the logic pins are 5-volt tolerant, so we can easily connect it to an Arduino or any 5V logic microcontroller without using a logic level converter.

**Technical Specifications**

| | |
|---|---|
| Frequency Range | 13.56 MHz ISM Band |
| Host Interface | SPI / I2C / UART |
| Operating Supply Voltage | 2.5 V to 3.3 V |
| Max. Operating Current | 13-26mA |

| | |
|---|---|
| Min. Current(Power down) | 10µA |
| Logic Inputs | 5V Tolerant |
| Read Range | 5 cm |

The RC522 module has a total of 8 pins that connect it to the outside world. The connections are as follows:




RC522 Pinout

Last Minute ENGINEERS.com

VCC supplies power to the module. This can be anywhere from 2.5 to 3.3 volts. You can connect it to the 3.3V output from your Arduino. But remember that connecting it to the 5V pin will probably destroy your module!
RST is an input for reset and power-down. When this pin goes low the module enters power-down mode. In which the oscillator is turned off and the input pins are disconnected from the outside world. Whereas the module is reset on the rising edge of the signal.
GND is the ground pin and needs to be connected to the GND pin on the Arduino.
IRQ is an interrupt pin that alerts the microcontroller when an RFID tag is in the vicinity.
MISO / SCL / Tx pin acts as master-in-slave-out when SPI interface is enabled, as serial clock when I2C interface is enabled and as serial data output when the UART interface is enabled.
MOSI (Master Out Slave In) is the SPI input to the RC522 module.
SCK (Serial Clock) accepts the clock pulses provided by the SPI bus master i.e. Arduino.
SS / SDA / Rx pin acts as a signal input when the SPI interface is enabled, as serial data when the I2C interface is enabled and as a serial data input when the UART interface is enabled. This pin is usually marked by encasing the pin in a square so that it can be used as a reference to identify other pins.

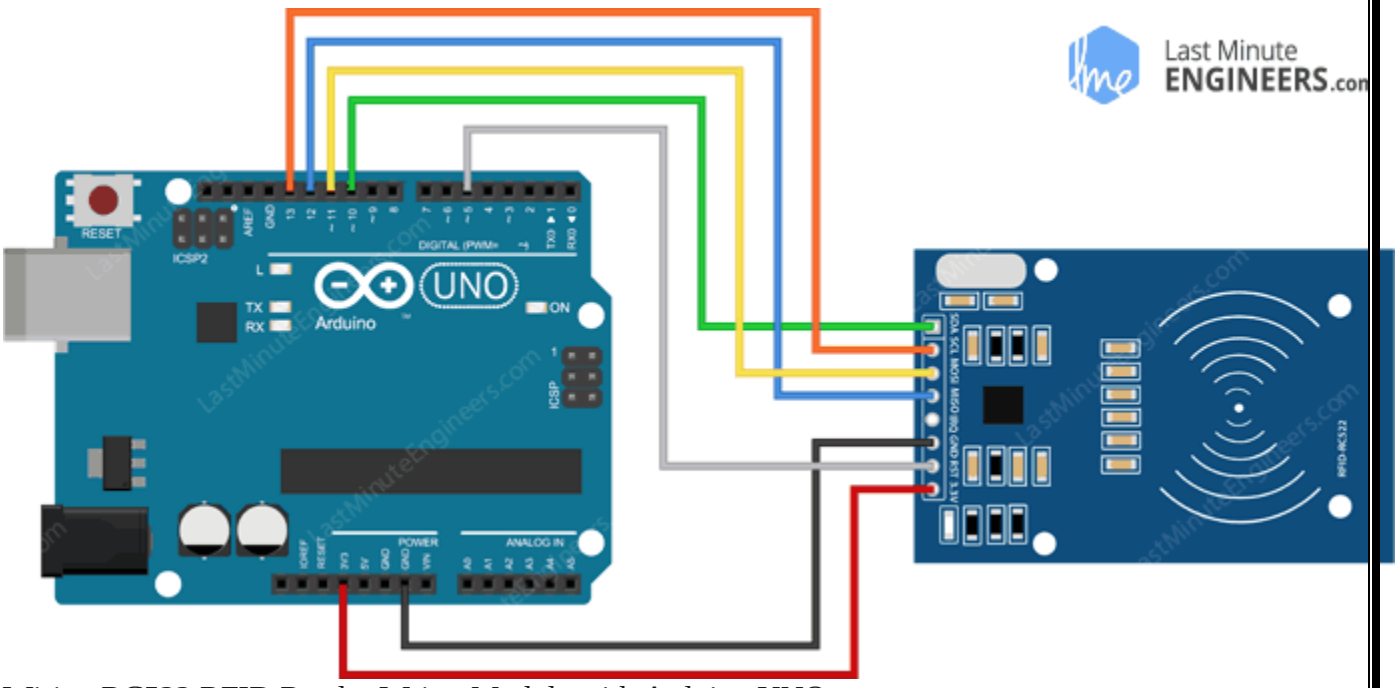
First connect the VCC pin on the module to 3.3V and the GND pin to ground on the Arduino. Pin RST can be connected to any digital pin on the Arduino. In our case, it is connected to digital pin #5. The IRQ pin is left unconnected because the Arduino library we are going to use does not support it.

Now we are left with the pins that are used for SPI communication. Since RC522 modules require a lot of data transfer, they will give the best performance when connected to the hardware SPI pins on the microcontroller.

Note that each Arduino board has different SPI pins that must be connected accordingly. Check the table below for quick understanding.

| | MOSI | MISO | SCK | CS |
|---|---|---|---|---|
| Arduino Uno | 11 | 12 | 13 | 10 |
| Arduino Nano | 11 | 12 | 13 | 10 |
| Arduino Mega | 51 | 50 | 52 | 53 |

If you are using a different Arduino than the boards mentioned above, please check the Arduino's official documentation before proceeding.




Wiring RC522 RFID Reader Writer Module with Arduino UNO

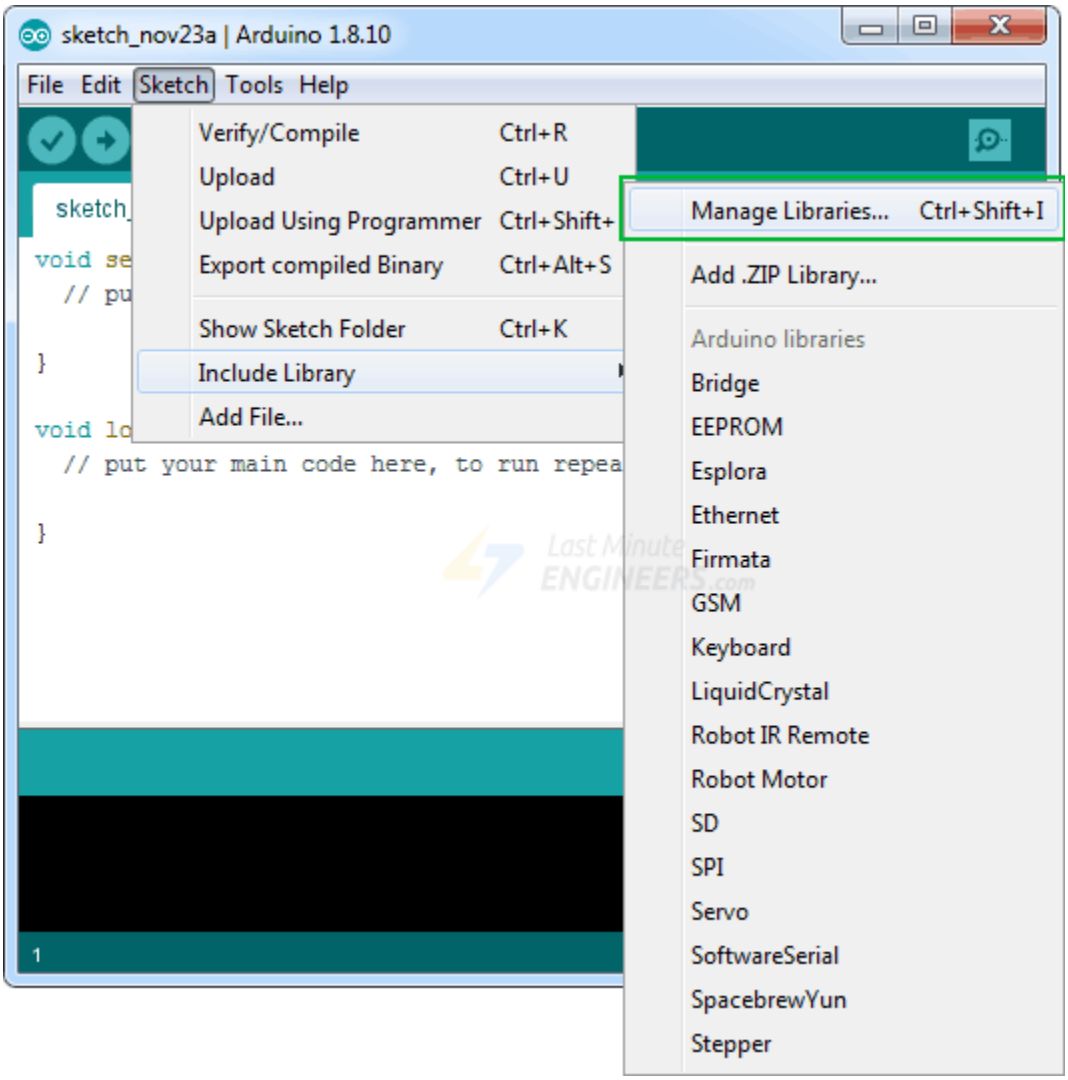
Once you have connected everything you are ready to go!

Library Installation

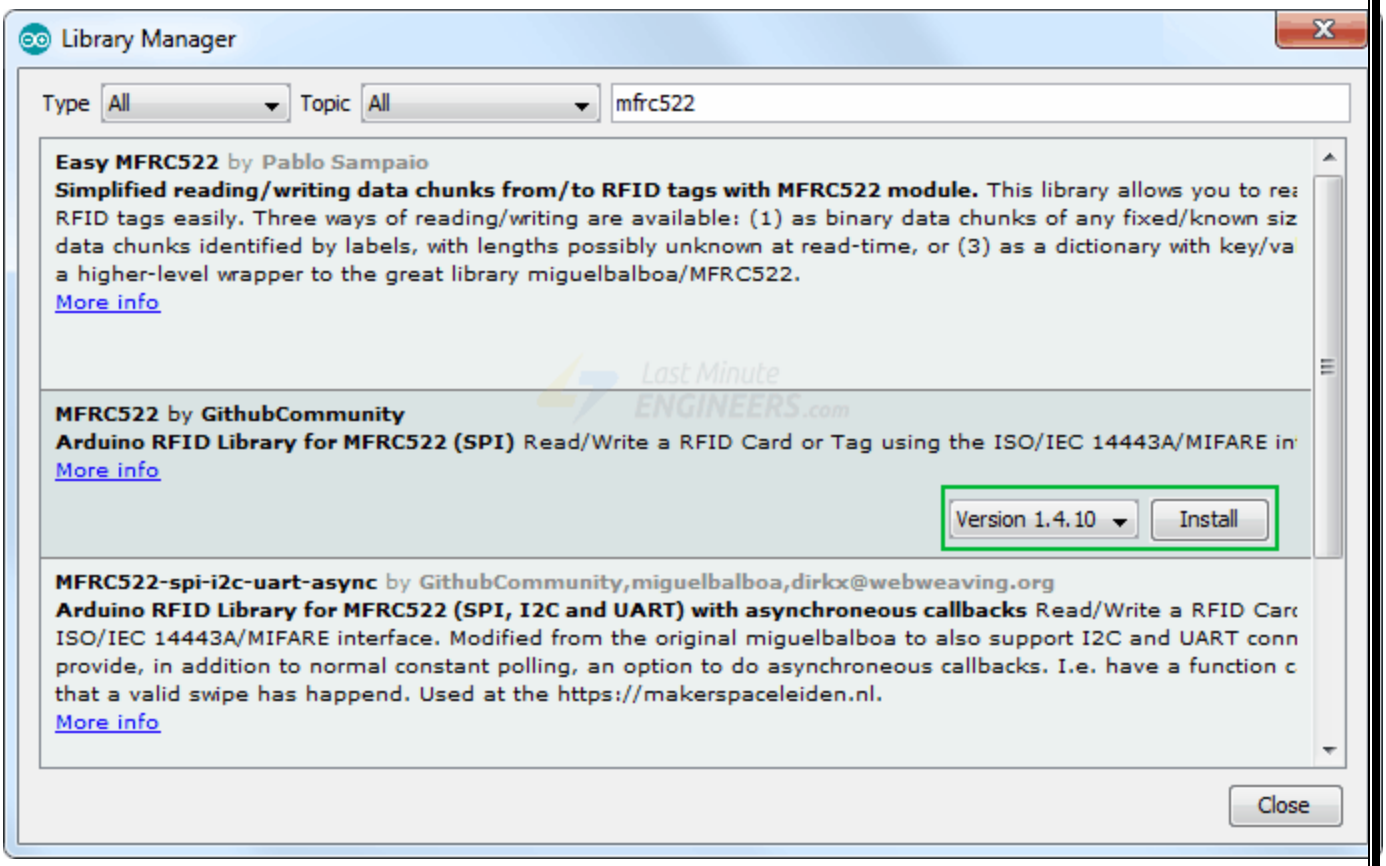
Communicating with an RC522 RFID module is a lot of work, but luckily for us there is a library called the MFRC522 library that makes reading and writing RFID tags simple.

This library is not included in the Arduino IDE, so you will need to install it first.

To install the library navigate to Sketch > Include Libraries > Manage Libraries… Wait for Library Manager to download the library index and update the list of installed libraries.




Filter your search by typing ‘mfrc522‘. Look for the library by GithubCommunity. Click on that entry, and then select Install.




## Arduino Code – Reading an RFID Tag

```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN A5

MFRC522 rfid(SS_PIN, RST_PIN); // Instance of the class

MFRC522::MIFARE_Key key;

// Init array that will store new NUID
byte nuidPICC[4];

void setup() {
  Serial.begin(9600);
  SPI.begin(); // Init SPI bus
  rfid.PCD_Init(); // Init MFRC522

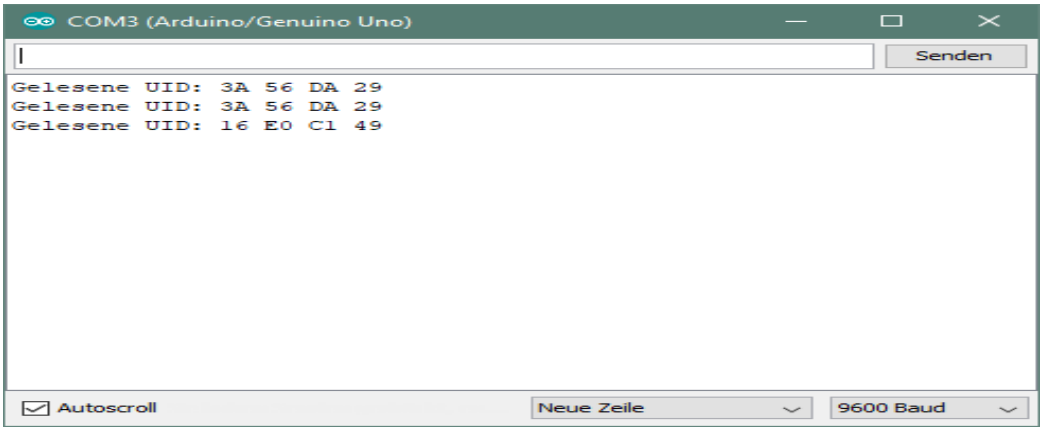
  for (byte i = 0; i < 6; i++) {
    key.keyByte[i] = 0xFF;
```

```
  }
}
void loop() {

  // Look for new cards
  if ( ! rfid.PICC_IsNewCardPresent())
    return;

  // Verify if the NUID has been readed
  if ( ! rfid.PICC_ReadCardSerial())
    return;
 for (byte i = 0; i < 4; i++) {
      nuidPICC[i] = rfid.uid.uidByte[i];
    }
  printHex(rfid.uid.uidByte, rfid.uid.size);
    Serial.println();
   rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}
void printHex(byte *buffer, byte bufferSize) {
  for (byte i = 0; i < bufferSize; i++) {
    Serial.print(buffer[i] < 0x10 ? " 0" : " ");
    Serial.print(buffer[i], HEX);
  }
}
```

**OUTPUT:**



**WEEK-5**

**AIM:** Write a program to monitor temperature and humidity using Arduino and Raspberry Pi.

**Hardware Requirements**:

1. Arduino UNO Board
2. DHT11 Temperature and Humidity Sensor(3 pins)
3. Jumper Wires
4. Bread Board

**Procedure:**

1. Connect pin 1 (on the left) of the sensor to +5V.

NOTE: If using a board with 3.3V logic like an Arduino Due connect pin 1

// to 3.3V instead of 5V!

2. Connect pin 2 of the sensor to whatever your DHTPIN is

3. Connect pin 4 (on the right) of the sensor to GROUND

4. Connect a 10K resistor from pin 2 (data) to pin 1 (power) of the sensor.

**Arduino Source Code:**

```
#include "DHT.h"
#define DHTPIN 2
//#define DHTTYPE DHT11   // DHT 11
#define DHTTYPE DHT11   // DHT 22  (AM2302), AM2321
//#define DHTTYPE DHT21   // DHT 21 (AM2301)
DHT dht(DHTPIN, DHTTYPE);
void setup() {
 Serial.begin(9600);
Serial.println(F("DHTxx test!"));
dht.begin();
}
void loop() {
 delay(2000);
```

```
  float h = dht.readHumidity();
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();
// Read temperature as Fahrenheit (isFahrenheit = true)
  float f = dht.readTemperature(true);
  if (isnan(h) || isnan(t) || isnan(f)) {
Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }
  float hif = dht.computeHeatIndex(f, h);
  float hic = dht.computeHeatIndex(t, h, false);
  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
  Serial.print(t);
  Serial.print(F("°C "));
  Serial.print(f);
  Serial.print(F("°F  Heat index: "));
  Serial.print(hic);
  Serial.print(F("°C "));
  Serial.print(hif);
  Serial.println(F("°F"));
}
```

**RaspberryPi Source Code**

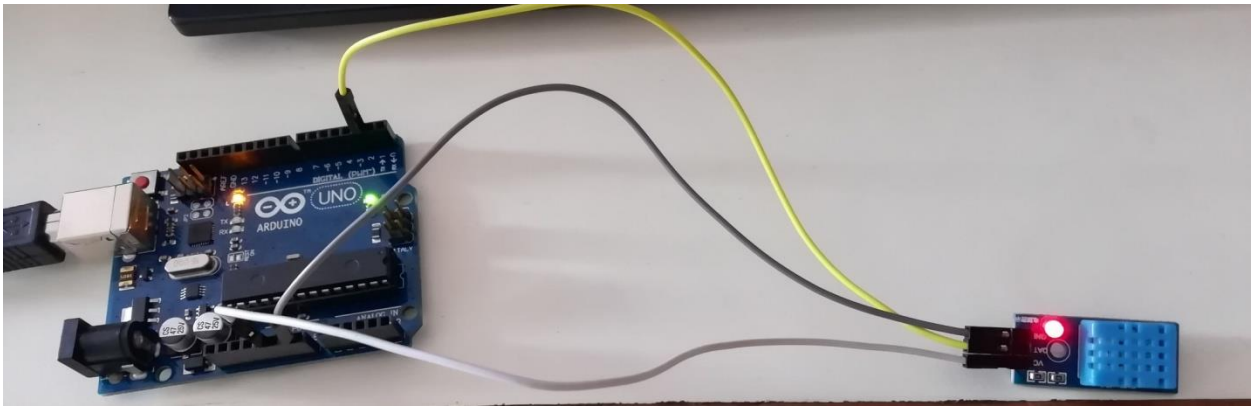
```
import time
import board
import adafruit_dht
dhtDevice = adafruit_dht.DHT11(board.D22,use_pulseio=False)

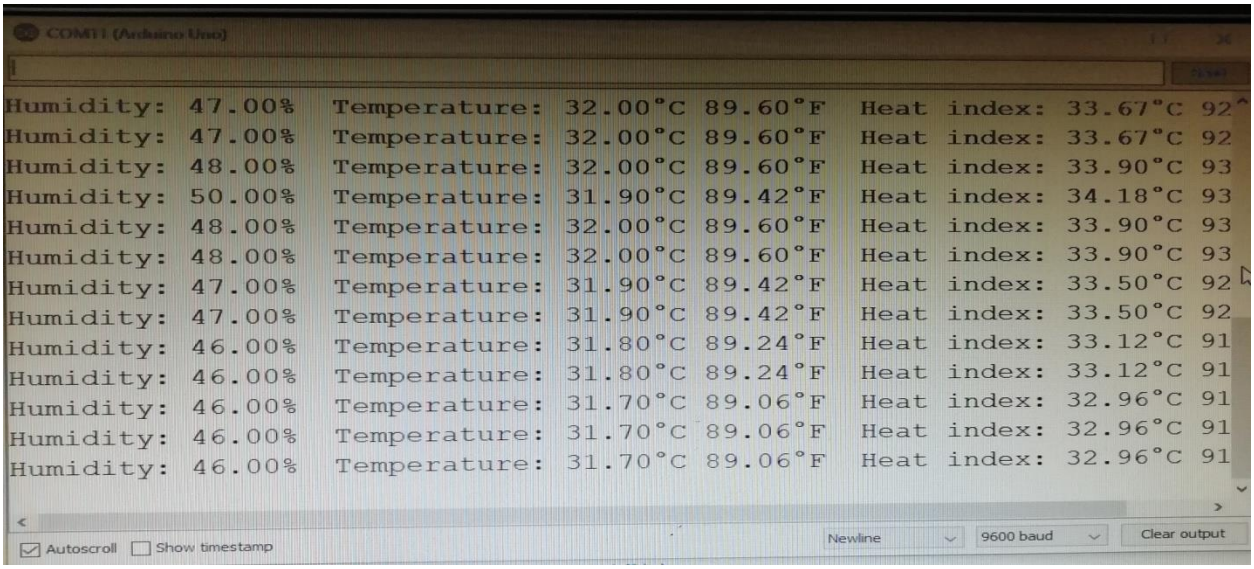
try:
    while True:
        # Print the values to the serial port
        temperature_c = dhtDevice.temperature
        temperature_f = temperature_c * (9 / 5) + 32
        humidity = dhtDevice.humidity
        print(
```

```
            "Temp:  {:.1f} C    Humidity: {}% ".format(
                temperature_c, humidity
            )
        )

except RuntimeError as error:
        # Errors happen fairly often, DHT's are hard to read, just keep
going
        print(error.args[0])
        time.sleep(2.0)
```

**OUTPUT:**



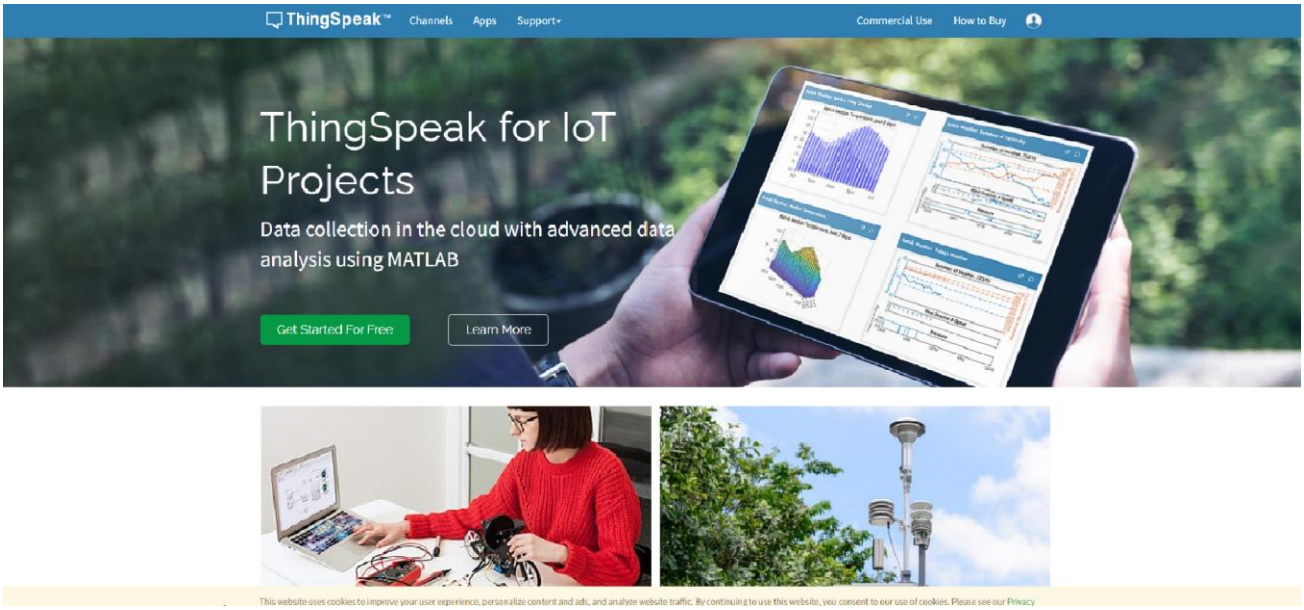



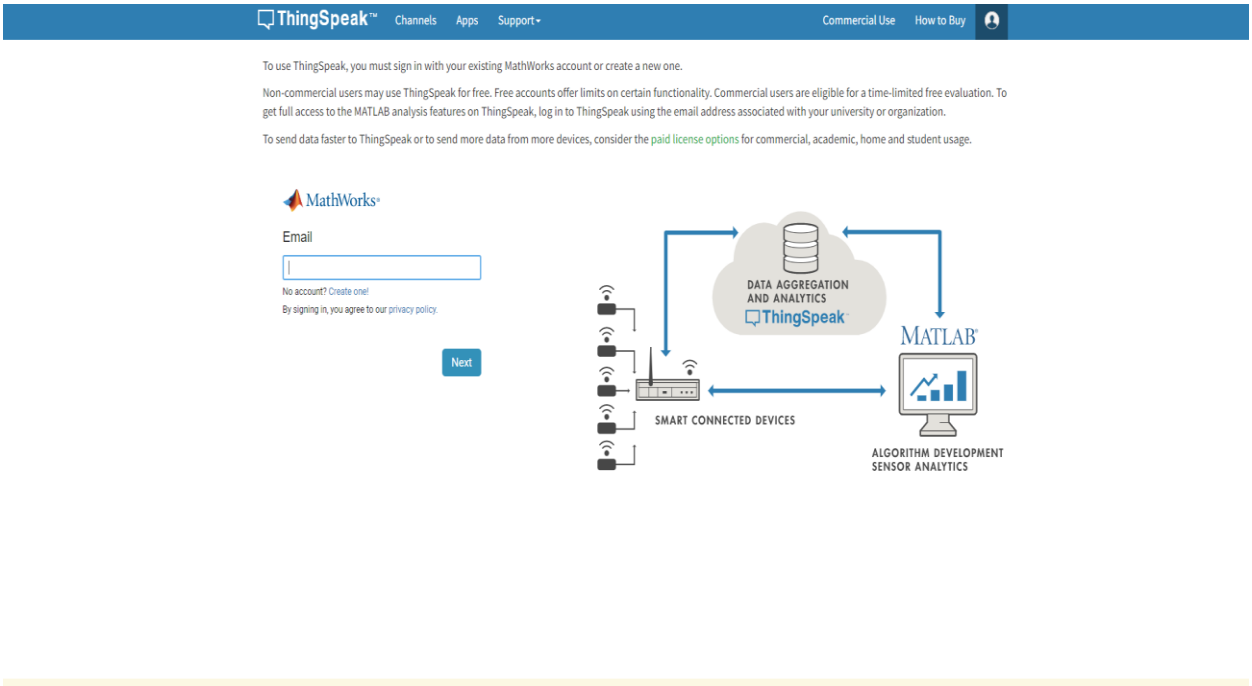
## WEEK-6

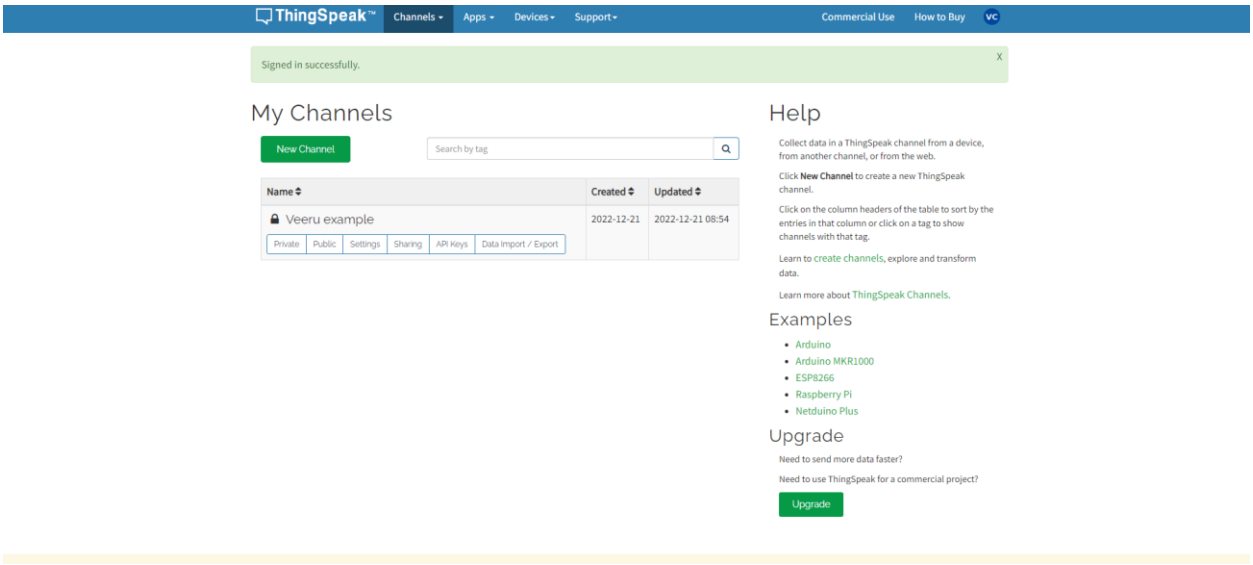
**Aim:** Write a program to monitor Sensor data using Arduino using IoT Cloud application.

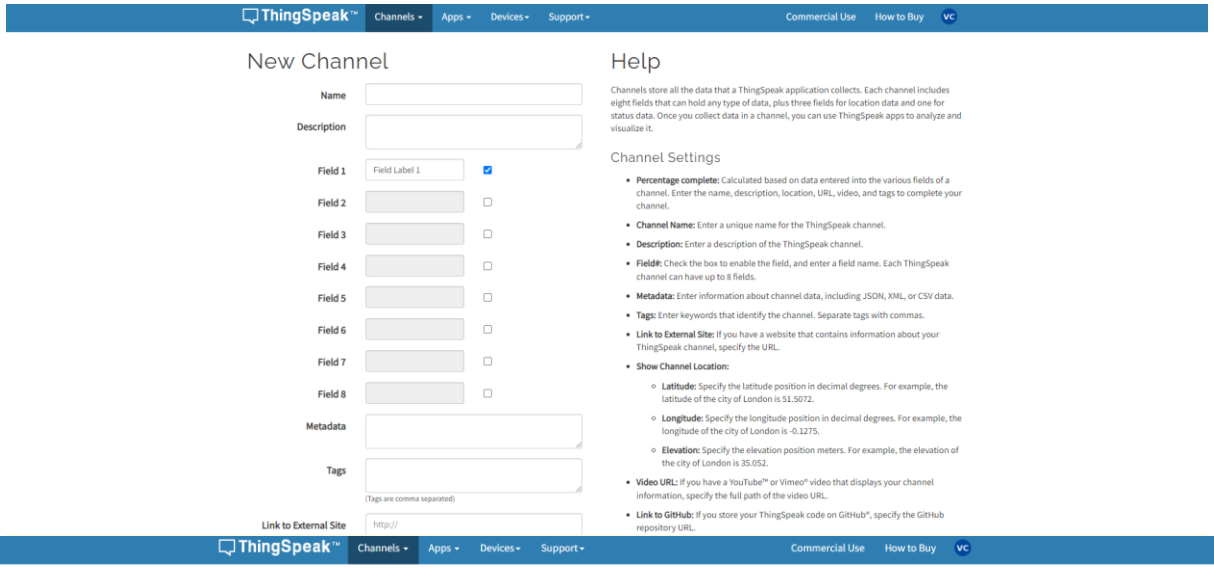
**IOT-Cloud Application:**

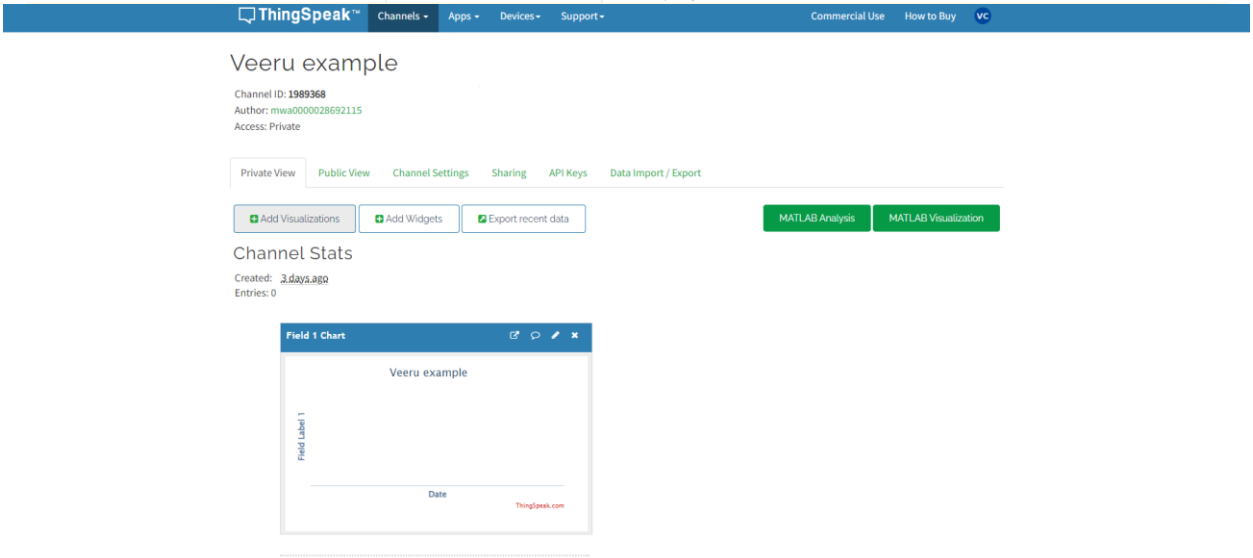
## THINGSPEAK

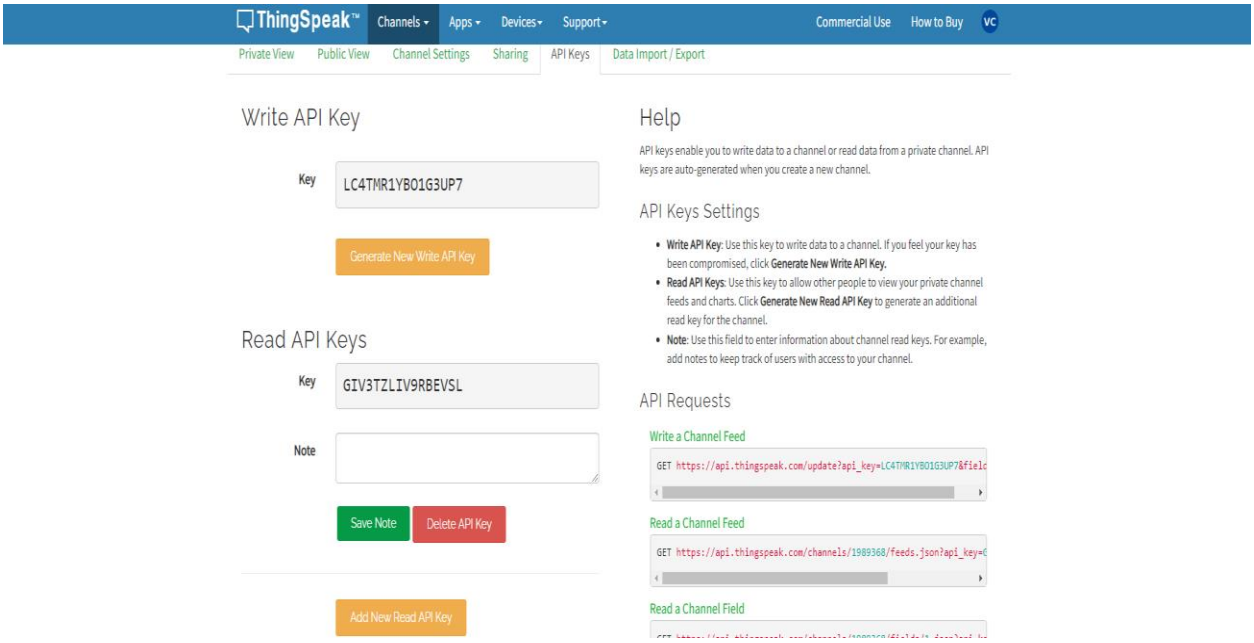



ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Signed in successfully.
My Channels
New Channel
Search by tag
Name
Created
Updated
Veeru example
2022-12-21
2022-12-21 08:54
Private
Public
Settings
Sharing
API Keys
Data Import / Export
Help
Collect data in a ThingSpeak channel from a device, from another channel, or from the web.
Click New Channel to create a new ThingSpeak channel.
Click on the column headers of the table to sort by the entries in that column or click on a tag to show channels with that tag.
Learn to create channels, explore and transform data.
Learn more about ThingSpeak Channels.
Examples
Arduino
Arduino MKR1000
ESP8266
Raspberry Pi
Netduino Plus
Upgrade
Need to send more data faster?
Need to use ThingSpeak for a commercial project?
Upgrade


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
New Channel
Name
Description
Field 1
Field Label 1
Field 2
Field 3
Field 4
Field 5
Field 6
Field 7
Field 8
Metadata
Tags
(Tags are comma separated)
Link to External Site
http://
Help
Channels store all the data that a ThingSpeak application collects. Each channel includes eight fields that can hold any type of data, plus three fields for location data and one for status data. Once you collect data in a channel, you can use ThingSpeak apps to analyze and visualize it.
Channel Settings
Percentage complete: Calculated based on data entered into the various fields of a channel. Enter the name, description, location, URL, video, and tags to complete your channel.
Channel Name: Enter a unique name for the ThingSpeak channel.
Description: Enter a description of the ThingSpeak channel.
FieldN: Check the box to enable the field, and enter a field name. Each ThingSpeak channel can have up to 8 fields.
Metadata: Enter information about channel data, including JSON, XML, or CSV data.
Tags: Enter keywords that identify the channel. Separate tags with commas.
Link to External Site: If you have a website that contains information about your ThingSpeak channel, specify the URL.
Show Channel Location:
Latitude: Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
Longitude: Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
Elevation: Specify the elevation position meters. For example, the elevation of the city of London is 35.052.
Video URL: If you have a YouTube™ or Vimeo® video that displays your channel information, specify the full path of the video URL.
Link to GitHub: If you store your ThingSpeak code on GitHub®, specify the GitHub repository URL.


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Veeru example
Channel ID: 1989368
Author: mwa0000028692115
Access: Private
Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export
Add Visualizations
Add Widgets
Export recent data
MATLAB Analysis
MATLAB Visualization
Channel Stats
Created: 3 days ago
Entries: 0
Field 1 Chart
Veeru example
Field Label 1
Date
ThingSpeak.com




**SOURCE CODE:**

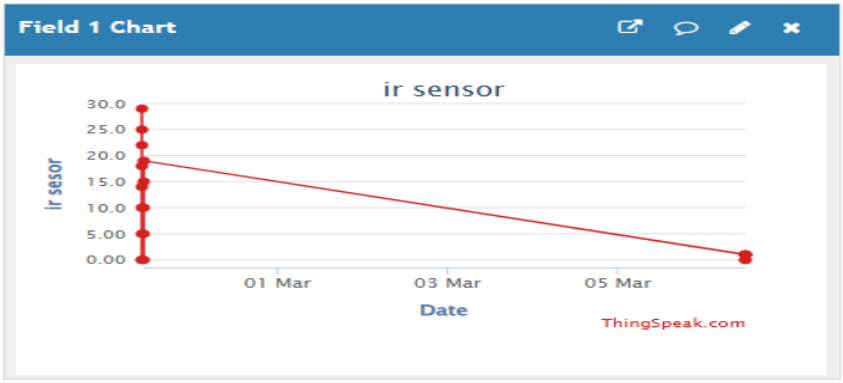
```
#include <ESP8266WiFi.h>
String apiKey = "xxx";
const char *ssid =  "xxx";
const char *pass =  "xxx";
const char* server = "api.thingspeak.com";
#define IRpin D2
WiFiClient client;
int value;
void setup()
{
Serial.begin(115200);
pinMode(IRpin, INPUT);
delay(1000);
Serial.println("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, pass);
while (WiFi.status() != WL_CONNECTED)
{
delay(2000);
Serial.print(".");
}
Serial.println("   ");
Serial.println("WiFi connected");
}
void loop(){
 value = digitalRead(IRpin);
Serial.println(value);
  if(value==0)
  {
```

```
  Serial.print("object detected");
  }
  else
  {
  Serial.print("no object detected");
  }
if (client.connect(server,80))
{
String postStr = apiKey;
postStr +="&field1=";
postStr += String(value);
postStr += "\r\n\r\n";

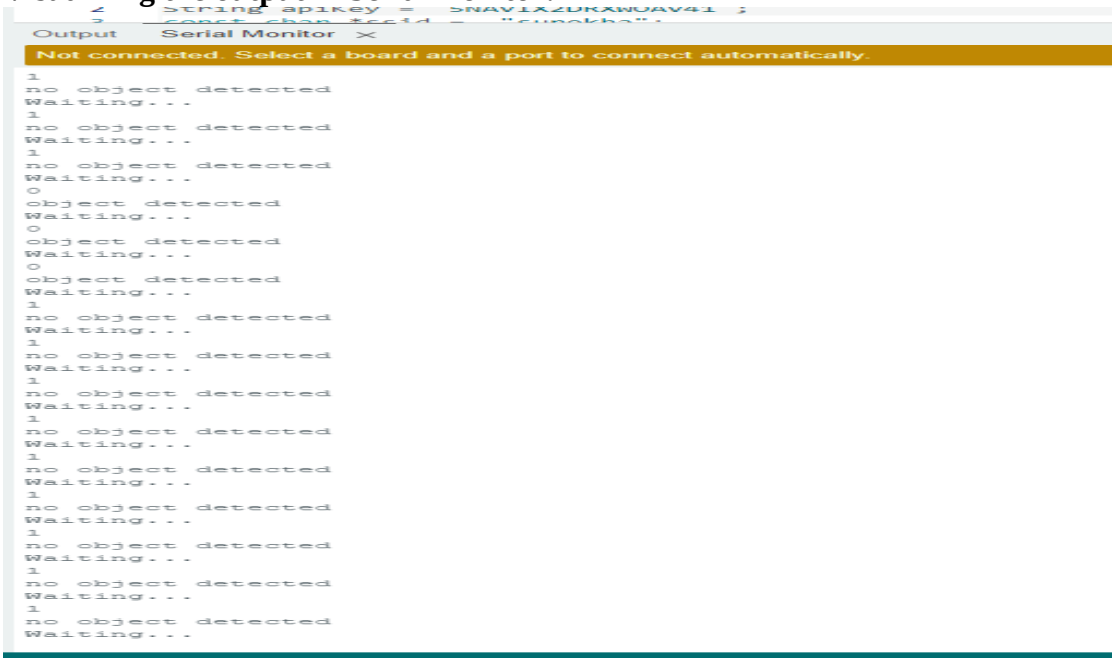
client.print("POST /update HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: "+apiKey+"\n");
client.print("Content-Type: application/x-www-form-urlencoded\n");
client.print("Content-Length: ");
client.print(postStr.length());
client.print("\n\n");
client.print(postStr);
client.stop();
Serial.println("Waiting...");
delay(1000); }}
```

**OUTPUT:**

**Uploading the data in CLOUD:**



**Visualizing the output in Serial monitor:**




## WEEK-7

**Aim:** Upload temperature and humidity data to the cloud using Arduino or Raspberry Pi.

**Hardware Requirements**:

1. NodeMCU ESP8266 Breakout Board
2. DHT-11 temperature and humidity sensor
3. Jumper wires
4. Bread board
5. WIFI Network

**Procedure:**

1. Download esp8266 Zip file->go to libraries->add Zip file
**2.** Connect Node MCU, Go to tools->change board to Node MCU esp8266 and port number
3. Connect DHT-11 temperature and Humidity sensor to Node MCU
4. Sign up to cloud->open THINGSPEAK>create channels->copy API key to the source code
5. SSID and password of your WIFI connection should be given in source code
6. Compile and upload the program and verify the temperature and humidity readings in serial monitor
7. Go to cloud and verify the temperature and humidity values in graph.

**SOURCE CODE:**

```
#include <DHT.h>
#include <DHT_U.h>
#include <ESP8266WiFi.h>
String apiKey = "ZYUP9R7N15OEBYRO";  //
const char *ssid =  "xxx";
```

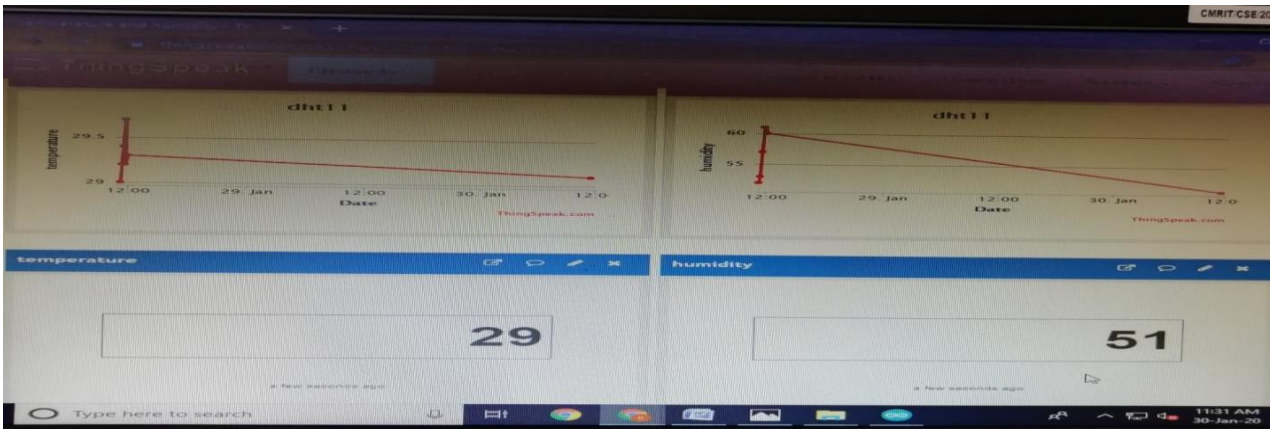
```
const char *pass =  "xxx";
const char* server = "api.thingspeak.com";
#define DHTPIN D3
DHT dht(DHTPIN, DHT11);
WiFiClient client;

 void setup()
{
  Serial.begin(115200);
  delay(1000);
  dht.begin();
  Serial.println("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(2000);
    Serial.print(".");
  }
Serial.println("");
Serial.println("WiFi connected");
}
void loop()
{
 float h = dht.readHumidity();
 float t = dht.readTemperature();
 if (isnan(h) || isnan(t))
 {
  Serial.println("Failed to read from DHT sensor!");
  return;
 }
if (client.connect(server,80))   //   "184.106.153.149" or api.thingspeak.com
{
  String postStr = apiKey;
  postStr +="&field1=";
  postStr += String(t);
  postStr +="&field2=";
  postStr += String(h);
  postStr += "\r\n\r\n";
client.print("POST /update HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: "+apiKey+"\n");
client.print("Content-Type: application/x-www-form-urlencoded\n");
client.print("Content-Length: ");
client.print(postStr.length());
```

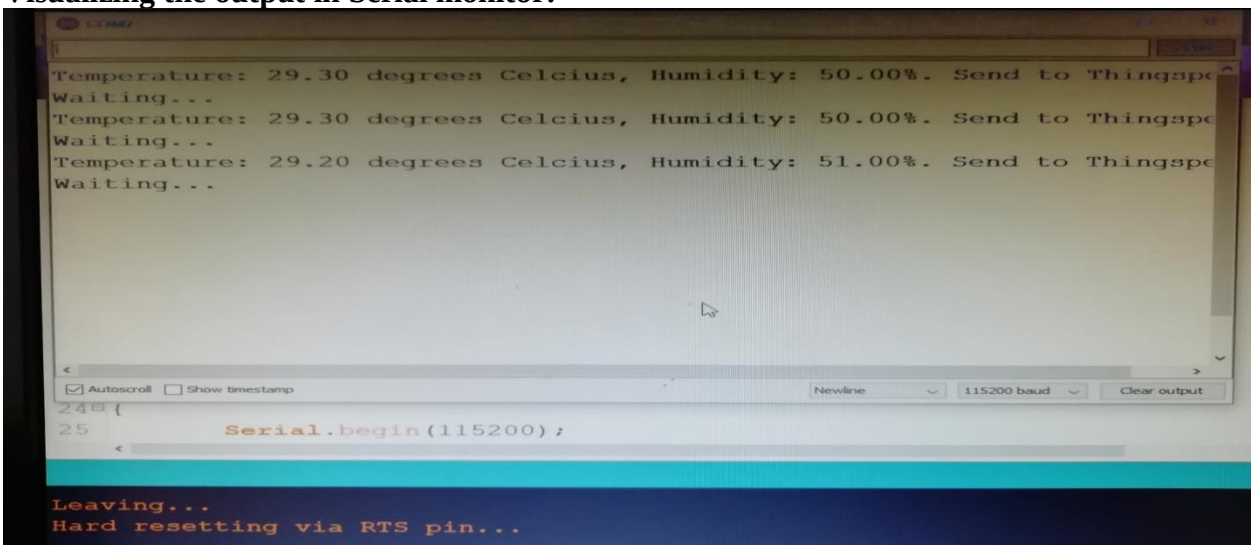
```
client.print("\n\n");
client.print(postStr);
Serial.print("Temperature: ");
Serial.print(t);
Serial.print(" degrees Celcius, Humidity: ");
Serial.print(h);
Serial.println("%. Send to Thingspeak.");
}
  client.stop();
  Serial.println("Waiting...");
  delay(1000);
}
}
```

**OUTPUT:**

**Uploading the data in CLOUD:**




**Visualizing the output in Serial monitor:**



## WEEK-8

**Aim:** Write a program on Arduino or Raspberry Pi to retrieve temperature and humidity data from the Cloud

**Hardware Requirements**:

1. NodeMCU ESP8266 Breakout Board
2. DHT-11 temperature and humidity sensor
3. Jumper wires
4. Bread board

**Procedure:**

1. Open up the Arduino IDE and head over to the library manager.

2. Install the DHT library (You can also install it by going to Sketch > Include Library > Manage Libraries, and search fort dht 8266 library).

**DHT sensor with 3 pins:**

1. Power supply 3.5V to 5.5V.

2. Data, Outputs both Temperature and Humidity through serial Data.

3. Ground, Connected to the ground of the circuit.

**Set up in source code:**

**1.** Set your Wi-Fi SSID and password.

**2.** Set the API Key

3. Save->Compile->upload->now us can visualize our data in cloud

**Source Code:**

```
#include "ThingSpeak.h"
#include <ESP8266WiFi.h>
#include<DHT.h>
const char ssid[] = "xxx";  // your network SSID (name)
const char pass[] = "xxx";   // your network password
int statusCode = 0;
WiFiClient  client;

//---------Channel Details---------//
unsigned long counterChannelNumber =  2846266;            // Channel ID
const char * myCounterReadAPIKey = "SXJMCVLJWIZMZLLF"; // Read API Key
```

```
const int FieldNumber1 = 1;  // The field you wish to read
const int FieldNumber2 = 2;  // The field you wish to read
//------------------------------//

void setup()
{
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  ThingSpeak.begin(client);
}

void loop()
{
  //----------------- Network -----------------//
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.print("Connecting to ");
    Serial.print(ssid);
    Serial.println(" ....");
    while (WiFi.status() != WL_CONNECTED)
    {
      WiFi.begin(ssid, pass);
      delay(5000);
    }
    Serial.println("Connected to Wi-Fi Succesfully.");
  }
  //--------- End of Network connection--------//

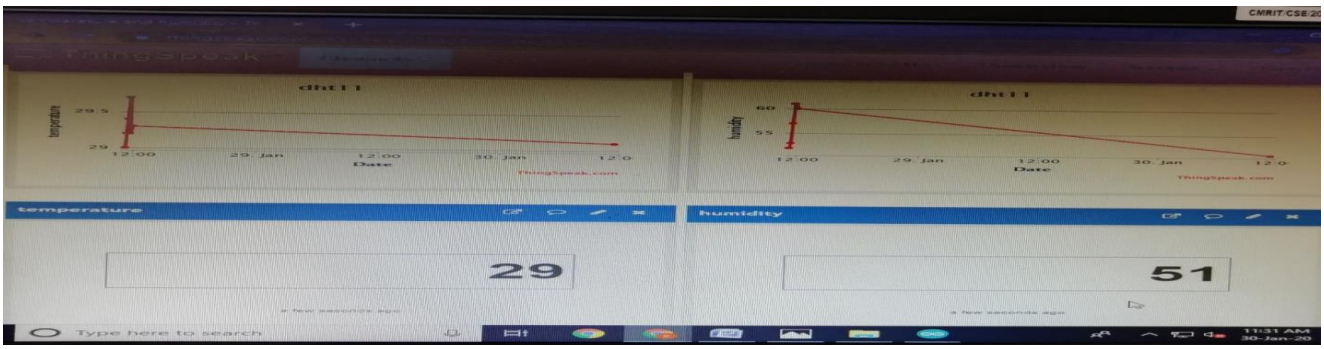
  //---------------- Channel 1 ----------------//
     long temp = ThingSpeak.readLongField(counterChannelNumber, FieldNumber1,
myCounterReadAPIKey);
  statusCode = ThingSpeak.getLastReadStatus();
  if (statusCode == 200)
  {
    Serial.print("Temperature: ");
    Serial.println(temp);
  }
  else
  {
    Serial.println("Unable to read channel / No internet connection");
  }
  delay(100);
  //-------------- End of Channel 1 -------------//

  //---------------- Channel 2 ----------------//
```

```
    long humidity = ThingSpeak.readLongField(counterChannelNumber, FieldNumber2, myCounterReadAPIKey);
  statusCode = ThingSpeak.getLastReadStatus();
  if (statusCode == 200)
  {
    Serial.print("Humidity: ");
    Serial.println(humidity);
  }
  else
  {
    Serial.println("Unable to read channel / No internet connection");
  }
  delay(100);
  //-------------- End of Channel 2 -------------//
}
```

**OUTPUT:**

**Uploading the data in CLOUD:**




**Visualizing the output in Serial monitor:**



**WEEK-9**

**AIM:** Write a program to create TCP Server on cloud using Arduino and Respond with humidity data to TCP Client when requested.

An IoT device can be made to communicate with a cloud or server using TCP/IP protocol without any hassle of network programming and network administration. In this project, an IoT device will be designed that could transmit sensor data to ThingSpeak Platform using the TCP/IP protocol.

The IoT device designed in this project is built using Arduino UNO. The Arduino is just a microcontroller board and cannot connect to an internet network on its own. For internet connectivity, the Arduino UNO is interfaced with ESP8266 module. The ESP8266 Wi-Fi Module is a self contained SOC with integrated TCP/IP protocol stack that can access to a Wi-Fi network. The ESP module allows the Arduino board to connect with a router and access internet network. The Arduino is programmed to communicate with the cloud platform.ThingSpeak over TCP/IP protocol. The Arduino can implement TCP/IP protocol by passing AT commands serially to the ESP8266 module.

The IoT device designed is a visitor counter as well as temperature and humidity monitor. For working as visitor counter, the IR sensors and Photodiodes are interfaced with the Arduino board. For working as temperature and humidity monitor, a DHT-11 sensor is interfaced with the Arduino board. The Arduino reads data from the sensors and send it to the ThingSpeak platform. A 0.6 inch 128 X 64 OLED is also interfaced with the Arduino which receives serial data from the board on I2C protocol and display the current temperature and humidity readings. The user can monitor the number of occupants in the house, temperature and humidity values from anywhere by accessing the ThingSpeak platform.

**The Arduino board controls all the functionalities of the IoT device like counting visitors, reading temperature and humidity values from DHT-11 sensor, displaying temperature and humidity data on OLED, implementing TCP/IP protocol, connecting with ThingSpeak platform and sending data to the cloud server.**

For this, the Arduino code is written and compiled using Arduino IDE.

**Software Required –**

- ThingSpeak server
- Arduino IDE

The IoT device that communicates with the ThingSpeak Cloud is built on Arduino UNO. The DHT-11 Sensor, IR sensor, Photodiodes, ESP8266 module and OLED module are interfaced with the Arduino board to make the IoT device.

**Arduino:** It is an Atmega 328 based controller board which has 14 GPIO pins, 6 PWM pins, 6 Analog inputs and on board UART, SPI and TWI interfaces. The Atmega 328 is the sitting MCU on the Arduino board. There are two GPIO pins (external interrupt pins INT0 and INT1) of Arduino board used to interface photodiodes, TX and RX pins (pins 3 and 2 respectively)are

used interface ESP module, SDA and SCL (pins 27 and 28 respectively) pins used to interface the OLED module.

**OLED Module –** The OLED module is used to display the temperature and humidity information as well as the count of the visitors (occupants) in the house. The module communicates with the Arduino board using I2C protocol. This is a two wire protocol. The module has four pins – SDA, SCL, VCC and GND. The VCC and Ground are connected to 5V DC and common ground respectively. The 5V DC can be supplied through a battery via 7805 voltage regulator IC. The SDA and SCL pins of the OLED module are connected to the SDA and SCL pins ((pins 27 and 28 respectively) of the Arduino board. The OLED are made up of carbon based organic materials. So unlike LCD displays, they do not require backlight and filters.

**ESP8266 Module –** The ESP8266 Wi-Fi Module is a self contained SOC with integrated TCP/IP protocol stack that can access to a Wi-Fi network. The ESP8266 is capable of either hosting an application or off loading all Wi-Fi networking functions from another application processor. Each ESP8266 module comes pre-programmed with an AT command set firmware. The Chip Enable and VCC pins of the module are connected to the 3.3 V DC while Ground pin is connected to the common ground. The chip enable pin is connected to VCC via a 10K pull up resistor. The RESET pin is left not connected. The Tx and Rx pins of the module are connected to the RX and TX pins of the Arduino UNO. The GPIO-0 pin of the module is connected to VCC through a 10K pull up resistor.

**IR Sensors –** An array of two IR LEDs is used in the circuit to detect entrance of any visitor. These IR LEDs are installed at the entrance of the house. An IR LED is a type of LED which emits light in the Infra-Red frequency range. The infra-red radiations are not visible to the human eye but can be seen by the lenses of a camera. Operationally, IR LEDs are not much different from normal LEDs. They also need a 3V DC for biasing and consume 20 mA current. They also need to be connected with pull-up resistor in a circuit. In the array, IR LEDs are connected with 220 ohm pull-up resistors.

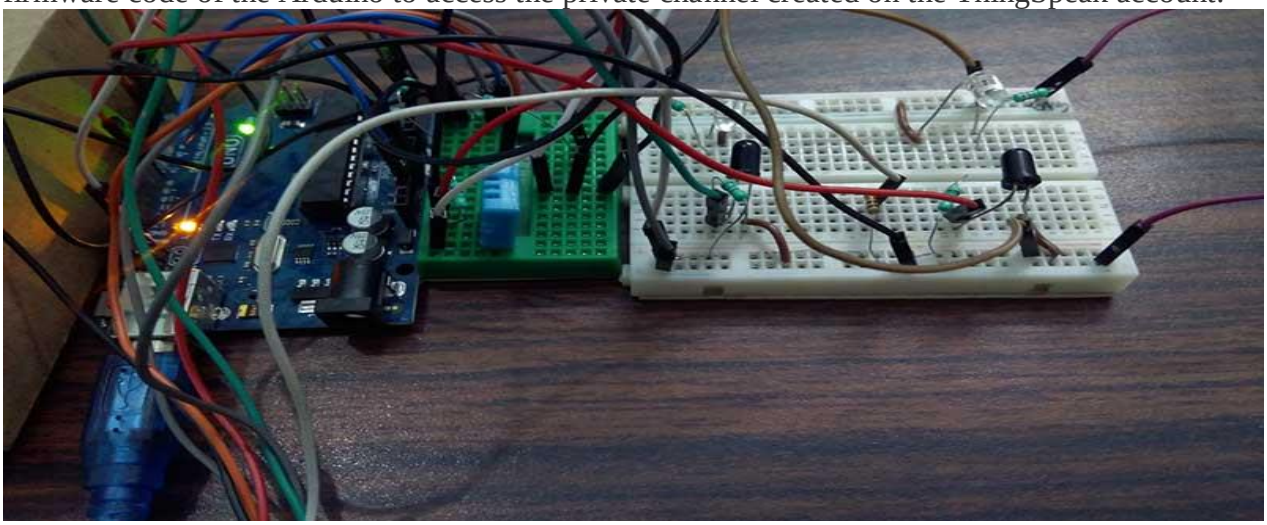
**Photodiodes –** The photodiodes are used as IR receivers in the circuit. A photodiode is a type of diode which gets forward biased when light is incident on it. It has a high resistance when no light is falling on it. When the intensity of light incident on it increases, it starts getting forward biased and current starts flowing through it. So, when light is incident on it, its resistance decreases and there is less voltage drop across it. When light is not incident on it, its resistance increases and there is higher voltage drop across it. The photodiode looks exactly like an LED and may have a dark blue or black film on the outer casing. The photodiodes are used in reverse bias configuration in the circuit. An array of two photodiodes installed in line with IR transmitters is used in the circuit. The photodiodes are interfaced at external interrupt pins INT0 and INT1 of the Arduino board.

**DHT-11 Sensor –** DHT-11 is a temperature and humidity sensor. The DHT11 sensor consists of two main components – one is Humidity sensing component and other is NTC temperature sensor (or Thermistor). The Thermistor is actually a variable resistor that changes its resistance with change in temperature. They both sense the temperature and humidity of area and give the output to the IC (which is placed on back side of sensor). The sensor has four pins – VCC, Ground, data Out and NC. The VCC and Ground pins are connected to the common VCC and Ground respectively. The Data Out pin of the sensor is connected to PD7 pin of the Arduino board via 10K pull-up resistor.

**ThingSpeak Server –** The ThingSpeak server is used to visualize the data received from the IoT device. The data is displayed in the form of graphs on the platform. The ThingSpeak generates

the read and write API key. The Write API key is used to write the data to the channel and read API channel is used to allow other people to view private channel feeds and charts. The data can also be saved on the platform for future reference.

In order to configure the ThingSpeak platform to access the data on it, first an **account** must be created on the platform. Then a channel must be created for the data on that account. It can be done by navigating to **channel window** and creating a new channel. The required information must be filled in the given form at the website so that the needed fields are created. For this project, there must be created three fields – Total Persons, Temperature and Humidity. These fields can then be checked live on the server. After saving the channel settings, a Write API key is generated which must be noted down. This Write API key is used in the firmware code of the Arduino to access the private channel created on the ThingSpeak account.



As the circuit is powered on the Arduino board starts reading data from the IR receivers and the DHT-11 sensor. The IR receivers are connected in series with variable resistors between VCC and ground in reverse bias configuration forming a voltage divider circuit. The output from the IR receivers (photodiodes) are drawn from the junction of cathode terminals of the IR receiver. Under normal conditions, the light emitted by the IR transmitters is continuously received by the photodiodes. This keeps the digital logic output of the photodiodes to HIGH. When a person enters the house, the light from the IR transmitters is blocked and the logical output from the photodiodes is switched to LOW. The IR receivers are connected at the external interrupt pins of the Arduino, so an interrupt is generated at the INT0 and INT1 pin of the Arduino when a person enters or exits the house. There are two pairs or IR transmitter and receiver used. The sequence in which the interrupts are generated at the INT0 and INT1 pins indicate whether a person has entered or exit from the house. Accordingly, the count of the current occupants of the house is increased or decreased.

The DHT11 Temperature and Humidity Sensor is a digital sensor with inbuilt capacitive humidity sensor and Thermistor. It relays a real-time temperature and humidity reading every 2 seconds. The sensor operates on 3.5 to 5.5 V supply and can read temperature between 0° C and 50° C and relative humidity between 20% and 95%. The DHT11 detects water vapors by measuring the electrical resistance between the two electrodes. The humidity sensing component is a moisture holding substrate with electrodes applied to the surface. When water vapors are absorbed by the substrate, ions are released by the substrate which increases the conductivity

between the electrodes. The change in resistance between the two electrodes is proportional to the relative humidity. Higher relative humidity decreases the resistance between the electrodes, while lower relative humidity increases the resistance between the electrodes.

**Source code:**

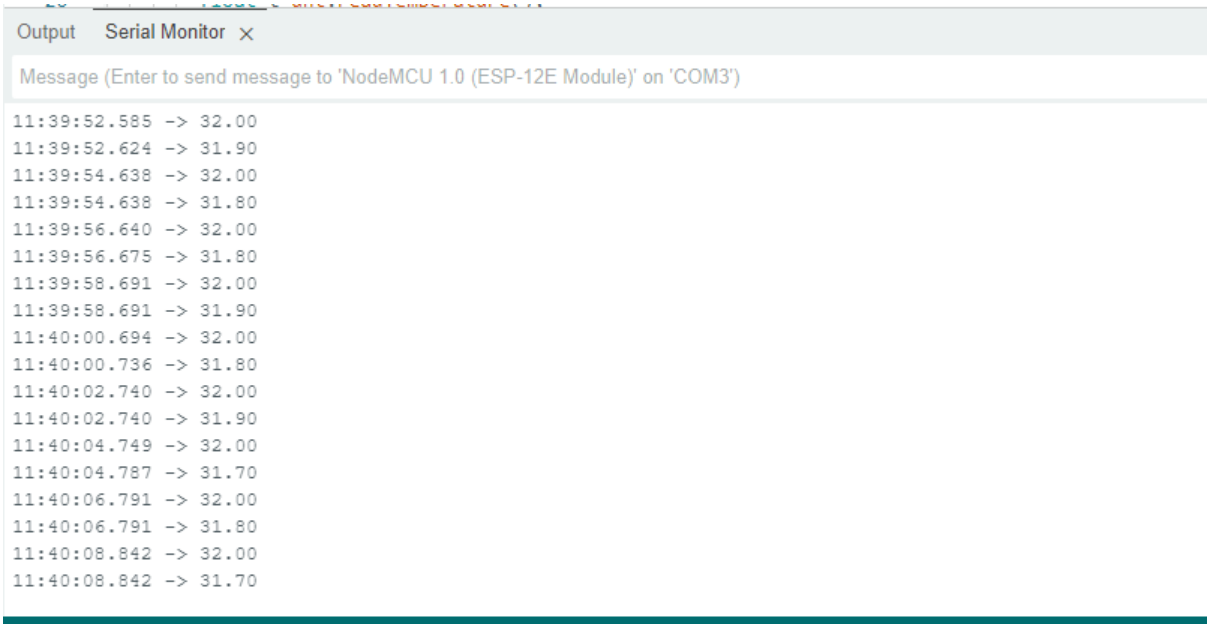
```
#include "ESP8266WiFi.h"
#include "DHT.h"
const char* ssid="Galaxy A21sE600";
const char* password ="zilh8480";
WiFiServer wifiServer(8080);
DHT dht(D3, DHT22);
void setup() {
  Serial.begin(115200);
  delay(1000);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting..");
  }
  Serial.print("Connected to WiFi. IP:");
  Serial.println(WiFi.localIP());
  wifiServer.begin();
  dht.begin();
}

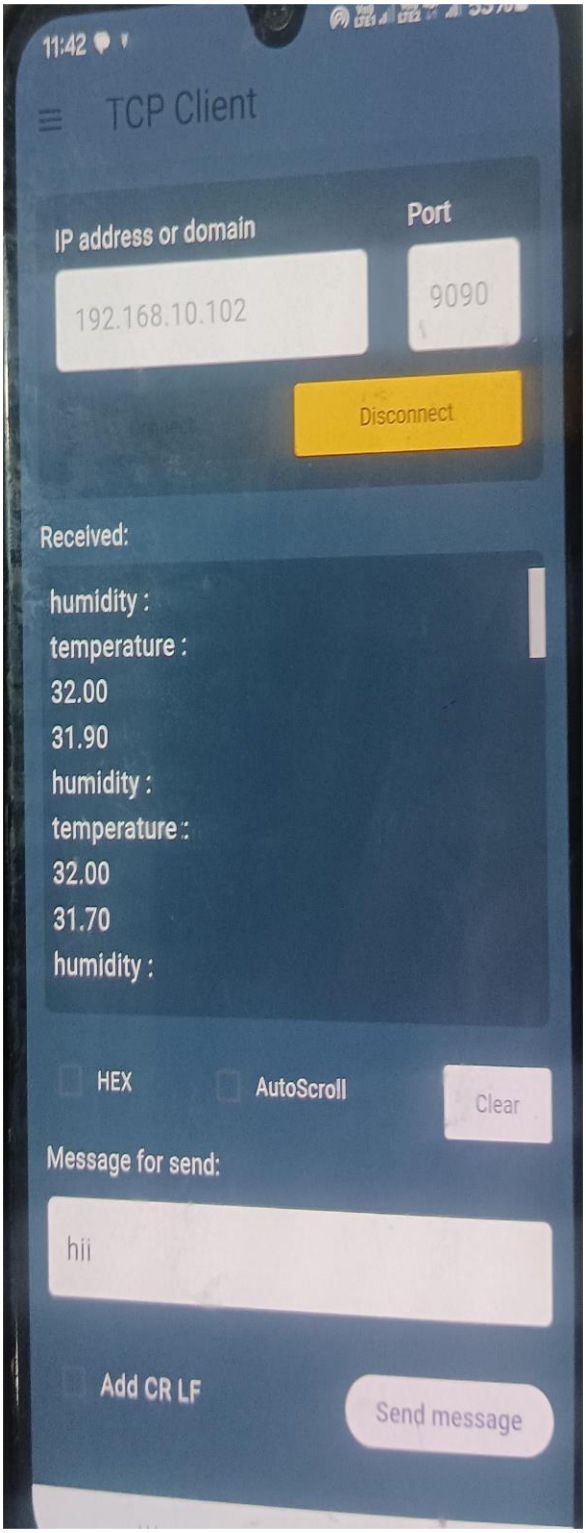
void loop() {
   WiFiClient client = wifiServer.available();
   if (client) {
     while (client.connected()) {
       while (client.available()>0) {
        float t=dht.readTemperature();
        float h = dht.readHumidity();
        client.print("humidity :");
        client.print("temperature :");
        client.println(h);
        Serial.println(h);
        client.println(t);
        Serial.println(t);
        delay(2000);
      }
    }
    client.stop();
```

```
    Serial.println("Client disconnected");
  }
}
```

**OUTPUT:**

```
11:40:57.483 -> 33.00
11:40:57.483 -> 31.50
11:40:59.479 -> 33.00
11:40:59.524 -> 31.50
11:41:01.506 -> 33.00
11:41:01.543 -> 31.60
11:41:29.425 -> Connecting..
11:41:30.459 -> Connecting..
11:41:31.446 -> Connecting..
11:41:32.451 -> Connecting..
11:41:32.451 -> Connected to WiFi. IP:192.168.10.102
```



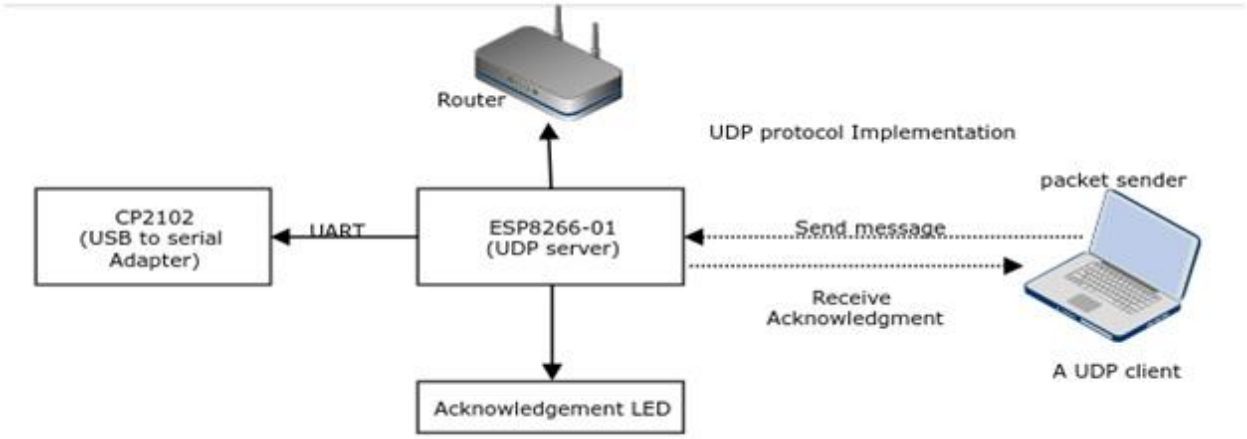

11:42
TCP Client
IP address or domain
Port
192.168.10.102
9090
Disconnect
Received:
humidity :
temperature :
32.00
31.90
humidity :
temperature :
32.00
31.70
humidity :
HEX
AutoScroll
Clear
Message for send:
hii
Add CR LF
Send message

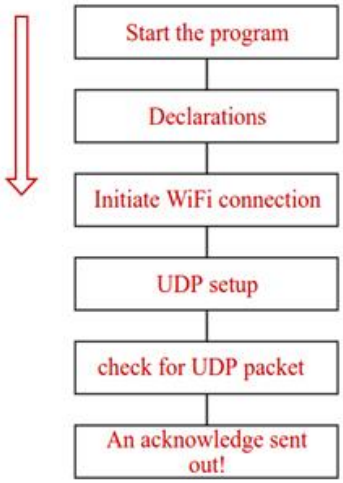
**WEEK-10**

**AIM:** Write a program to create UDP Server on cloud using Arduino and Respond with humidity data to UDP Client when requested.

The UDP protocol has a small overhead of 8 bytes which makes it more suitable for use in the Internet of Things. In this project, the application of UDP protocol in IoT will be demonstrated. In this project, an ESP8266 Wi-Fi modem will be configured as UDP server and a laptop will be used as UDP Client. Both Client and server will be co-located communicating through same Wi-Fi router so, the ESP board will act as a local serverThe ESP module working as server checks for the UDP packet received from the client on a particular port. When a valid packet is arrived, an acknowledge packet is sent back to the client to the same port it has been sent out. In response to receiving packet and sending acknowledgement, the ESP modem switches on an LED as visual indication of successful Client-Server Communication. This application is very useful as it demonstrates server/client communication over UDP protocol.

It can be said that the IOT device designed in this experiment is a simple UDP server with LED indicator. It is designed by interfacing an LED with the ESP-8266 Wi-Fi module. The Wi-Fi module as well as LED light are powered continuously with the help of a USB to Serial Converter. The Wi-Fi module needs to loaded with a firmware that could receive data over UDP protocol and respond with an acknowledgement. The Arduino UNO is used to flash the firmware code on the ESP8266 module. The ESP module can also be flashed with code using a FTDI converter like CP2102. The firmware itself is written in the Arduino IDE.






**Software Required –**

• ThingSpeak server

• Arduino IDE

ESP8266 board needs to be loaded with the firmware code. In this tutorial, the firmware code is written using Arduino IDE. It is loaded to the ESP8266 board using the Arduino UNO. A generic ESP8266 board is used in this project. This board does not have any bootstrapping resistors, no voltage regulator, no reset circuit and no USB-serial adapter. The ESP8266 module operates on 3.3 V power supply with current greater than or equal to 250 mA. So, CP2102 USB to serial adapter is used to provide 3.3 V voltage with enough current to run ESP8266 reliably in every situation. The ESP8266 Wi-Fi Module is a self contained SOC with integrated TCP/IP protocol stack that can access to a Wi-Fi network. The ESP8266 is capable of either hosting an application or off loading all Wi-Fi networking functions from another application processor. Each ESP8266 module comes pre-programmed with an AT command set firmware.

The Chip Enable and VCC pins of the module are connected to the 3.3 V DC while Ground pin is connected to the common ground. The chip enable pin is connected to VCC via a 10K pull up resistor. The RESET pin is connected to the ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default. The Tx and Rx pins of the module are connected to the RX and TX pins of the Arduino UNO. The GPIO-0 pin of the module is connected to ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default. Write the firmware code in the Arduino IDE and connect the Arduino board with the PC via USB cable. Open Arduino IDE and go to Tools->Port and select the Arduino board (Arduino UNO). It may look like /dev/ttyABM0 (Arduino/Genuino Uno). Select the correct port name. The port name can be different in different IDE setups. Then Open Serial monitor in the Arduino IDE by navigating to Tools->Serial Monitor and set the baud rate to 115200 bauds per second. Pass 'AT' and 'AT+GMR'

commands to test the connection between the Arduino and ESP-01 module. Try different settings for the 'Line ending' option of the serial monitor like Both NL & CR. Try different combinations until the ESP module starts interacting correctly with the serial monitor.

**SOURCE CODE:**

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <DHT.h>
const char* ssid = "Galaxy A21sE600";
const char* password = "zilh8480";
const char* udpAddress = "192.168.87.225";
const int udpPort = 1234;
#define DHTPIN D3
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
WiFiUDP udp;
void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(2000);
    Serial.print(".");
  }
  Serial.println();
  Serial.println("WiFi connected.");
  dht.begin();
}
void loop() {
  delay(2000);
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" °C\tHumidity: ");
  Serial.print(humidity);
  Serial.println(" %");
  Serial.println("Sending data over UDP...");
```

```
  udp.beginPacket(udpAddress, udpPort);
  udp.print("Temperature: ");
  udp.print(temperature);
  udp.print(" °C, Humidity: ");
  udp.print(humidity);
  udp.println(" %");
  udp.endPacket();
  Serial.println("Data sent over UDP.");
}
```

**OUTPUT:**

